combinatorial
binomial-inversion . . . 1
catalan . . . 1
lucas . . . 1
permutation-combination . . . 1
dynamic-programming
bounded-subset-sum . . . 1
divide-and-conquer . . . 2
knuth . . . 2
extra
divide-and-conquer . . . 2
offset-array . . . 2
permutation-int . . . 2
pragmas . . . 2
random . . . 2
submasks . . . 2
sum-over-subsets . . . 2
sum-over-supersets . . . 2
ternary-search . . . 2
time . . . 2
geometry
circle . . . 2
li-chao-tree . . . 3
line-container . . . 3
line . . . 3
point . . . 3
polar-sort . . . 4
polygon . . . 4
ray . . . 4
segment . . . 4
graph
binary-lifting . . . 4
centroid-decomposition . . . 4
articulations . . . 5
block-cut-tree . . . 5
bridge-tree . . . 5
bridges . . . 5
cycle . . . 5
dominator-tree . . . 5
steiner-tree . . . 5
tarjan . . . 6
bellman-ford . . . 6
floyd-warshall . . . 6
johnson . . . 6
dinic . . . 6
min-cost-k-flow . . . 7
heavy-light-decomposition . . . 7
hierholzer . . . 7
leaf-peeling . . . 8
link-cut-tree . . . 8
hopcroft-karp . . . 8
hungarian . . . 8
kuhn . . . 9
maximum-clique . . . 9
two-sat . . . 9
math
berlekamp-massey . . . 9
fast-fourier-transform . . . 10
fast-walsh-hadamard-transform . . . 10
modulo-transform-technique . . . 10
number-theoretic-transform . . . 10
gaussian-elimination . . . 11
josephus . . . 11
matrix . . . 11
nim . . . 11
xor-basis . . . 11
number-theory
floor-ceil . . . 12
floor-sum . . . 12
harmonic-lemma . . . 12
mobius . . . 12
chinese-remainder-theorem . . . 12
extended-gcd . . . 12
mod-log . . . 12
mod-pow . . . 12
multi-extended-gcd . . . 12
linear-sieve . . . 12
miller-rabin . . . 12
pollard-rho . . . 12
segmented-sieve . . . 13
totient . . . 13
square-root
decomposition . . . 13
mo-3d . . . 13
mo . . . 13
string
aho-corasick . . . 13
deterministic-finite-automaton . . . 14
dynamic-hasher . . . 14
hasher . . . 14
longest-prefix-suffix . . . 14
manacher . . . 14
palindromic-tree . . . 14
suffix-array . . . 14
suffix-automaton . . . 15
tree-suffix-array . . . 15
trie . . . 16
z-array . . . 16
structs
binary-indexed-tree . . . 16
range-binary-indexed-tree . . . 16
sparse-binary-indexed-tree . . . 16
cover-tree . . . 17
k-cover-tree . . . 17
coprime-set . . . 17
interval-set . . . 17
range-set . . . 17
venice-set . . . 18
disjoint-set-union . . . 18
dynamic-connectivity . . . 18
rollback-disjoint-set-union . . . 18
dynamic-merge-sort-tree . . . 18
merge-sort-tree . . . 18
hash-table . . . 19
ordered-set . . . 19
iterative-segment-tree . . . 19
segment-tree-beats . . . 19
segment-tree . . . 20
sparse-table . . . 20
implicit-treap . . . 20
treap . . . 21
wavelet-matrix . . . 21
theory
burnside-lemma . . . 21
exponent-reduction-euler-theorem . . . 21
flow-bipartite-graphs-reductions . . . 21
johnson-algorithm-potential-method . . . 22
manhattan-to-chebyshev . . . 22
mobius-inversion . . . 23
stars-and-bars . . . 23
vandermonde-identity . . . 23

combinatorial

binomial-inversion

```cpp
// Binomial Inversion: transforms subset group counts into exact
  frequency counts
// Time: O(n^2) | Space: O(n)
// Formula: exact[i] = sum{j=i}^{n} (-1)^(j-i) * C(j, i) * subsets[j]
vector<int> binomial_inversion(const vector<int> &subsets) {
  int n = subsets.size() - 1;
  vector<int> exact(n + 1);
  for (int i = 0; i <= n; ++i) {
    int count = 0;
    for (int j = i; j <= n; ++j) {
      int term = (C(j, i) * subsets[j]) % mod;
      if ((j - i) & 1) {
        count = (count - term + mod) % mod;
      } else {
        count = (count + term) % mod;
      }
    }
    exact[i] = count;
  }
  return exact;
}
```

catalan

```cpp
// Catalan (Ballot Theorem): number of grid paths from (0,0) to (l,r)
  staying x >= y at all times
// Time: O(1) | Space: O(1)
// Note: requires C(n, k) | also counts valid parenthesis prefixes (l
  open, r close)
int catalan(int l, int r) {
  if ((l < 0 or r < 0) or l < r) return 0;
  return (C(l + r, l) - C(l + r, l + 1) + mod) % mod;
}
```

lucas

```cpp
int lucas(int n, int k) {
  if (k == 0 or k == n) return 1;
  if (k > n) return 0;
  return (lucas(n / mod, k / mod) * C(n % mod, k % mod)) % mod;
}
```

permutation-combination

```cpp
int P(int n, int k) {
  if (k < 0 or n < k) return 0;
  return fact[n] * invf[n - k] % mod;
}

int C(int n, int k) {
  if (k < 0 or n < k) return 0;
  return P(n, k) * invf[k] % mod;
}
```

dynamic-programming

bounded-subset-sum

```cpp
// Bounded Subset Sum: maximizes subset sum without exceeding the
  target
// Time: O(N * max(W)) | Space: O(max(W))
// Note: efficient for huge targets with small weights
int bounded_subset_sum(vector<int> &w, int target) {
  int k = 0, s = 0;
  while (k < w.size() and s + w[k] <= target) s += w[k++];

  if (k == w.size() or s == target) return s;

  int bound = *max_element(w.begin(), w.end());
  int offset = target - bound;

  vector<int> prev(2 * bound + 1, -1);
  prev[s - offset] = k;

  for (int i = k; i < w.size(); ++i) {
    vector<int> curr = prev;

    // insert
    for (int x = 0; x < bound; ++x) if (prev[x] != -1) {
      curr[x + w[i]] = max(curr[x + w[i]], prev[x]);
    }

    // remove
    for (int x = 2 * bound; x > bound; --x) if (curr[x] > prev[x]) {
      for (int j = max(0ll, prev[x]); j < curr[x]; ++j) {
        curr[x - w[j]] = max(curr[x - w[j]], j);
      }
    }

    swap(prev, curr);
  }

  for (int x = bound; x >= 0; --x) if (prev[x] != -1) {
    return x + offset;
  }
  return 0;
}
```

divide-and-conquer

```cpp
int divide_and_conquer() {
  vector<int> prev(n + 1, inf);
  vector<int> curr(n + 1, inf);
  auto compute = [&](auto &self, int l, int r, int opt_l, int opt_r)
    -> void {
    if (l > r) return;
    int m = (l + r) / 2;
    curr[m] = inf;
    int opt = opt_r;
    for (int i = max(opt_l, m); i <= min(opt_r, n - 1); ++i) if
      (prev[i + 1] != inf) {
      int val = cost(m, i) + prev[i + 1];
      if (val < curr[m]) {
        curr[m] = val;
        opt = i;
      }
    }
    self(self, l, m - 1, opt_l, opt);
    self(self, m + 1, r, opt, opt_r);
  };
  for (int i = 1; i <= n; ++i) {
    curr[i] = cost(i, n);
  }
  for (int i = 2; i <= k; ++i) {
    swap(prev, curr);
    compute(compute, 1, n, 1, n);
  }
  return curr[1];
}
```

knuth

```cpp
int knuth() {
  vector<vector<int>> val(n + 1, vector<int>(n + 1));
  vector<vector<int>> opt(n + 1, vector<int>(n + 1));
  for (int i = 1; i <= n; ++i) {
    val[i][i] = cost(i, i);
    opt[i][i] = i;
  }
  for (int len = 2; len <= n; ++len) {
    for (int l = 1, r = len; r <= n; ++l, ++r) {
      val[l][r] = inf;
      opt[l][r] = opt[l][r - 1];
      for (int k = opt[l][r - 1]; k <= min(r - 1, opt[l + 1][r]);
        ++k) {
        int curr = val[l][k] + val[k + 1][r] + cost(l, r);
        if (curr < val[l][r]) {
          val[l][r] = curr;
          opt[l][r] = k;
        }
      }
    }
  }
  return val[1][n];
}
```

extra

divide-and-conquer

```cpp
vector<int> divide_and_conquer() {
  int n, k;
  vector<int> a(n + 1);
  vector<int> pref(n + 2), suf(n + 2);
  vector<int> ans(k + 1);
  auto rec = [&](auto &rec, int low, int high, vector<tuple<int, int,
    int>> &q) -> void {
    if (low > high or q.empty()) return;
    int mid = (low + high) / 2;
    pref[mid + 1] = 0;
    suf[mid] = 0;
    for (int i = mid; i >= low; --i) pref[i] = a[i] + pref[i + 1];
    for (int i = mid + 1; i <= high; ++i) suf[i] = a[i] + suf[i - 1];
    vector<tuple<int, int, int>> ql, qr;
    for (auto [l, r, i] : q) {
      if (r < mid) ql.push_back({l, r, i});
      else if (l > mid) qr.push_back({l, r, i});
      else ans[i] = pref[l] + suf[r];
    }
    rec(rec, low, mid - 1, ql);
    rec(rec, mid + 1, high, qr);
  };
  return ans;
}
```

offset-array

```cpp
template <typename T>
struct offset_array {
  int l;
  vector<T> a;

  offset_array(int l, int r, T x = T()): l(l), a(r - l, x) {}

  T &operator[](int i) {
    return a[i - l];
  }

  const T &operator[](int i) const {
    return a[i - l];
  }
};
```

permutation-int

```cpp
int permutation_to_int(vector<int> &p) {
  int mask = 0;
  int i = 0, r = 0;
  for (int x : p) {
    r = r * ++i + __builtin_popcountll(mask & -(1LL << x));
    mask |= 1LL << x;
  }
  return r;
}
```

pragmas

```cpp
#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

random

```cpp
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<int> dist(l, r);
```

submasks

```cpp
void submasks(int mask) {
  for (int submask = mask; submask > 0; submask = mask & (submask -
    1)) {
    process(submask);
  }
  process(0);
}
```

sum-over-subsets

```cpp
vector<int> sum_over_subsets(int n, vector<int> f) {
  int full = (1 << n) - 1;
  for (int i = 0; i < n; ++i) {
    int bit = 1 << i;
    for (int mask = 0; mask <= full; ++mask) if (mask & bit) {
      f[mask] += f[mask ^ bit];
    }
  }
  return f;
}
```

sum-over-supersets

```cpp
vector<int> sum_over_supersets(int n, vector<int> f) {
  int full = (1 << n) - 1;
  for (int i = 0; i < n; ++i) {
    int bit = 1 << i;
    for (int mask = 0; mask <= full; ++mask) if (not (mask & bit)) {
      f[mask] += f[mask | bit];
    }
  }
  return f;
}
```

ternary-search

```cpp
int ternary_search(int low, int high, bool is_max) {
  while (high - low >= 3) {
    int mid1 = low + (high - low) / 3;
    int mid2 = high - (high - low) / 3;

    if (is_max == (fun(mid1) < fun(mid2))) low = mid1;
    else high = mid2;
  }

  int ans = low;
  for (int i = low + 1; i <= high; ++i) if (is_max == (fun(ans) <
    fun(i))) ans = i;
  return ans;
}

float ternary_search(float low, float high, bool is_max) {
  for (int itr = 1; itr <= 100; ++itr) {
    float mid1 = low + (high - low) / 3.0;
    float mid2 = high - (high - low) / 3.0;

    if (is_max == (fun(mid1) < fun(mid2))) low = mid1;
    else high = mid2;
  }

  return (low + high) / 2.0;
}
```

time

```python
from datetime import date, timedelta

# Convert 'HH:MM:SS' string to total seconds since midnight
def time_to_seconds(time_str: str) -> int:
  h, m, s = map(int, time_str.split(':'))
  return h * 3600 + m * 60 + s

# Convert total seconds back to 'HH:MM:SS' format
def seconds_to_time(seconds: int) -> str:
  h = seconds // 3600
  m = (seconds % 3600) // 60
  s = seconds % 60
  return f"{h:02d}:{m:02d}:{s:02d}"

# Fast parse 'YYYY-MM-DD' directly into integers
def parse_date(date_str: str) -> tuple:
  y, m, d = map(int, date_str.split('-'))
  return y, m, d

# Check if a given date is Saturday or Sunday
def is_weekend(year: int, month: int, day: int) -> bool:
  # weekday() returns 0 for Monday up to 6 for Sunday
  return date(year, month, day).weekday() >= 5

# Get the new (year, month, day) after adding x days
def add_days(year: int, month: int, day: int, x_days: int) -> tuple:
  new_date = date(year, month, day) + timedelta(days=x_days)
  return new_date.year, new_date.month, new_date.day

# Calculate the exact number of days between two dates
def days_between(y1: int, m1: int, d1: int, y2: int, m2: int, d2:
  int) -> int:
  date1 = date(y1, m1, d1)
  date2 = date(y2, m2, d2)
  return (date2 - date1).days

# Fast check if a year is a leap year (pure math, no imports)
def is_leap_year(year: int) -> bool:
  return (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
```

geometry

circle

```cpp
point incircle(point a, point b, point c) {
  T ab = dist(a, b);
  T bc = dist(b, c);
  T ac = dist(a, c);
  T p = ab + bc + ac;
  if (p == 0) return {};
  return (a * bc + b * ac + c * ab) / p;
}

point circumcircle(point a, point b, point c) {
  point v = b - a;
  point w = c - a;
  T h = cross(v, w);
  if (h == 0) return {};
  return a + perp(v * norm(w) - w * norm(v)) / (2.0 * h);
}

vector<point> circle_line_inter(point o, T r, line l) {
  T s = l.side(o);
  T d2 = (s * s) / norm(l.v);
  T h2 = r * r - d2;
  if (h2 < 0) return {};
  point p = l.proj(o);
  if (h2 == 0) return {p};
  point h = l.v * sqrt(h2 / norm(l.v));
  return {p - h, p + h};
}

vector<point> circle_segment_inter(point o, T r, point a, point b) {
  if (a == b) {
    if (norm(a - o) == r * r) return {a};
    return {};
  }
  vector<point> l = circle_line_inter(o, r, line(a, b));
  vector<point> h;
  for (point p : l) if (in_disk(a, b, p)) h.push_back(p);
  return h;
}

T circle_inter_area(point o1, T r1, point o2, T r2) {
  T d = dist(o1, o2);
  if (d >= r1 + r2) return 0.0;
  if (d <= abs(r1 - r2)) return pi * min(r1, r2) * min(r1, r2);
  T h1 = 2.0 * acos((d * d + r1 * r1 - r2 * r2) / (2.0 * d * r1));
  T h2 = 2.0 * acos((d * d + r2 * r2 - r1 * r1) / (2.0 * d * r2));
  T a1 = 0.5 * r1 * r1 * (h1 - sin(h1));
  T a2 = 0.5 * r2 * r2 * (h2 - sin(h2));
  return a1 + a2;
}

vector<point> circle_inter(point o1, T r1, point o2, T r2) {
```

```
  point v = o2 - o1;
  T d2 = norm(v);
  if (d2 == 0) return {};
  T op = (d2 + r1 * r1 - r2 * r2) / 2.0;
  T h2 = r1 * r1 - (op * op) / d2;
  if (h2 < 0) return {};
  point p = o1 + v * (op / d2);
  if (h2 == 0) return {p};
  point h = perp(v) * sqrt(h2 / d2);
  return {p - h, p + h};
}
```

li-chao-tree

```
// Li Chao Tree: dynamic line & segment container for maximum convex
  hull trick
// Time: line insert/query O(log U) | segment insert O(log^2 U) |
  Space: O(N log U)
// Note: computes max(ax + b) | for min: insert (-a, -b) and take
  -query(x)
struct li_chao_tree {
private:
  struct line {
    int a, b;

    int operator()(int x) const {
      return a * x + b;
    }
  };

  struct node {
    line f;
    int l, r;
  };

  int min_x, max_x;
  vector<node> tree;

  int create_node() {
    tree.push_back({{0, -inf}, 0, 0});
    return tree.size() - 1;
  }

  int insert_line(line f, int p, int l, int r) {
    if (p == 0) p = create_node();
    int m = l + (r - l) / 2;
    bool is_l = f(l) > tree[p].f(l);
    bool is_m = f(m) > tree[p].f(m);
    if (is_m) swap(f, tree[p].f);
    if (l == r) return p;
    if (is_l != is_m) tree[p].l = insert_line(f, tree[p].l, l, m);
    else tree[p].r = insert_line(f, tree[p].r, m + 1, r);
    return p;
  }

  int insert_segment(line f, int ql, int qr, int p, int l, int r) {
    if (p == 0) p = create_node();
    if (ql <= l and r <= qr) return insert_line(f, p, l, r);
    int m = l + (r - l) / 2;
    if (ql <= m) tree[p].l = insert_segment(f, ql, qr, tree[p].l, l,
      m);
    if (qr > m) tree[p].r = insert_segment(f, ql, qr, tree[p].r, m +
      1, r);
    return p;
  }

  int query(int x, int p, int l, int r) {
    if (p == 0) return -inf;
    int y = tree[p].f(x);
    if (l == r) return y;
    int m = l + (r - l) / 2;
    if (x <= m) y = max(y, query(x, tree[p].l, l, m));
    else y = max(y, query(x, tree[p].r, m + 1, r));
    return y;
  }

public:
  li_chao_tree(int min_x, int max_x): min_x(min_x), max_x(max_x) {
    tree.push_back({{0, -inf}, 0, 0});
    create_node();
  }

  void insert_line(int a, int b) {
    insert_line({a, b}, 1, min_x, max_x);
  }

  void insert_segment(int a, int b, int l, int r) {
    insert_segment({a, b}, l, r, 1, min_x, max_x);
  }

  int query(int x) {
    return query(x, 1, min_x, max_x);
  }
};
```

line-container

```
// Line Container: dynamic convex hull trick for maximum
// Time: insert/query O(log N) | Space: O(N)
// Note: computes max(ax + b) | change '<' to '>' in line operator
  for min
struct line {
  mutable int a, b, i;

  bool operator<(const line &o) const {
    return tie(a, b) < tie(o.a, o.b);
  }

  bool operator<(const int x) const {
    return i < x;
  }
};

struct line_container : multiset<line, less<>> {
  const int inf = 1e18;

  int div(int x, int y) {
    return (x / y) - (((x ^ y) < 0) & x % y != 0);
  }

  bool isect(iterator s, iterator t) {
    if (t == end()) return s->i = inf, false;
    if (s->a == t->a) s->i = -inf;
    else s->i = div(t->b - s->b, s->a - t->a);
    return s->i >= t->i;
  }

  void insert(int a, int b) {
    auto z = multiset::insert({a, b, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() and isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() and (--x)->i >= y->i) isect(x,
      erase(y));
  }

  int max(int x) {
    auto l = lower_bound(x);
    return l->a * x + l->b;
  }
};
```

line

```
struct line {
  point v;
  T c;

  line(point v, T c): v(v), c(c) {}

  line(T a, T b, T c): v({b, -a}), c(c) {}

  line(point p, point q): v(q - p), c(cross(q, p)) {}

  T side(point p) {
    return cross(v, p) - c;
  }

  float dist(point p) {
    return abs(side(p)) / sqrt(norm(v));
  }

  bool comp(point p, point q) {
    return dot(v, p) < dot(v, q);
  }

  // Projection: calculates the orthogonal projection of point p onto
    the line
  // Note: translates p perpendicularly to the line (assumes perp is
    counter-clockwise)
  point proj(point p) {
    return p - (perp(v) * side(p)) / norm(v);
  }

  // Reflection: finds the mirror image of point p across the line
  // Note: projects p and extends it by the same distance
  point refl(point p) {
    return p - (2.0 * perp(v) * side(p)) / norm(v);
  }
};

vector<point> line_inter(line l, line r) {
  if (cross(l.v, r.v) == 0) return {};
  return {(l.c * r.v - r.c * l.v) / cross(l.v, r.v)};
}

vector<point> line_segment_inter(line l, point a, point b) {
  T sa = l.side(a);
  T sb = l.side(b);
  if (sign(sa) * sign(sb) > 0) return {};
  if (sa == 0 and sb == 0) {
    vector<point> p = {a, b};
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    return p;
  }
  return {(a * sb - b * sa) / (sb - sa)};
}
```

point

```
#define x real()
#define y imag()

using T = int;
using point = complex<T>;

namespace std {
  bool operator<(const point &a, const point &b) {
    return pair(a.x, a.y) < pair(b.x, b.y);
  }
}

istream &operator>>(istream &is, point &p) {
  T r, i; is >> r >> i;
  p = {r, i};
  return is;
}

int sign(T n) {
  if (n == 0) return 0;
  return n < 0 ? -1 : +1;
}

float dist(point a, point b) {
  return sqrt(norm(a - b));
}

T dot(point p, point q) {
  return (conj(p) * q).x;
}

T cross(point p, point q) {
  return (conj(p) * q).y;
}

T orient(point a, point b, point c) {
  return cross(b - a, c - a);
}

point perp(point p) {
  return {-p.y, p.x};
}

// Directed Angle: shortest angle from vector v to vector w
// Note: returns radians in [-pi, pi], negative result means w is to
  the right
float angle(point v, point w) {
  return atan2(cross(v, w), dot(v, w));
}

float closest_pair(vector<point> p) {
  int n = p.size();
  sort(p.begin(), p.end());
  auto compare = [](const point &a, const point &b) {
    if (a.y != b.y) return a.y < b.y;
    return a.x < b.x;
  };
  multiset<point, decltype(compare)> active(compare);
  active.insert(p[0]);
  float d = inf;
  int l = 0;
  for (int i = 1; i < n; ++i) {
    while (l < i and p[i].x - p[l].x >= d) {
      active.erase(active.find(p[l]));
      l++;
    }
    auto begin = active.lower_bound({p[i].x - d, p[i].y - d});
    auto end = active.upper_bound({p[i].x + d, p[i].y + d});
    for (auto itr = begin; itr != end; ++itr) {
      float c = dist(*itr, p[i]);
      d = min(d, c);
    }
    active.insert(p[i]);
  }
  return d;
}
```

polar-sort

```
// Is Upper: determines which half-plane a point lies in relative to
  ray v
// Note: returns 1 for (0, pi] (upper half and opposite ray), 0 for
  (pi, 2pi] (lower half, ray v and origin)
bool is_upper(point p, point v) {
  T c = cross(v, p);
  T d = dot(v, p);
  return c > 0 or (c == 0 and d < 0);
}

// Polar Sort: sorts points counter-clockwise starting from the
  opposite of reference ray v
// Time: O(N log N) | Space: O(1)
// Note: colinear points sorted by distance
void polar_sort(vector<point> &p, point o = {0, 0}, point v = {1, 0})
  {
  sort(p.begin(), p.end(), [&](point a, point b) {
    a = a - o, b = b - o;
    bool ha = is_upper(a, v), hb = is_upper(b, v);
    if (ha != hb) return ha < hb;
    T c = cross(a, b);
    if (c != 0) return c > 0;
    return norm(a) < norm(b);
  });
}
```

polygon

```
float area(point a, point b, point c) {
  return abs(cross(b - a, c - a)) / 2.0;
}

float area(const vector<point> &p) {
  T a = 0;
  int n = p.size();
  for (int i = 0; i < n; ++i) a += cross(p[i], p[(i + 1) % n]);
  return abs(a) / 2.0;
}

// Pick's Theorem: calculates the area of a simple polygon with
  integer vertices
// Time: O(1) | Space: O(1)
// Note: i is the number of interior points, b is the number of
  boundary points
float picks_theorem(int i, int b) {
  return i + (b / 2.0) - 1;
}

// Boundary Points: calculates the total number of integer points on
  the boundary of a polygon
// Time: O(N log M) | Space: O(1)
// Note: relies on the property that the number of integer points on
  a segment is gcd(dx, dy)
int boundary_points(const vector<point> &g) {
  int c = 0, n = g.size();
  for (int i = 0; i < n; ++i) {
    point p = g[i], q = g[(i + 1) % n];
    c += gcd(p.x - q.x, p.y - q.y);
  }
  return c;
}

// Winding Number: computes how many times a polygon wraps around a
  point
// Time: O(N) | Space: O(1)
// Note: returns non-zero for inside, zero for outside or boundary
int winding_number(const vector<point> &g, point a) {
  int w = 0, n = g.size();
  for (int i = 0; i < n; ++i) {
    point p = g[i], q = g[(i + 1) % n];
    if (on_segment(p, q, a)) return 0;
    int d = (q.y >= a.y) - (p.y >= a.y);
    if (d * orient(p, q, a) > 0) w += d;
  }
  return w;
}

vector<point> convex_hull(vector<point> g, bool col) {
  sort(g.begin(), g.end());
  g.erase(unique(g.begin(), g.end()), g.end());
  int n = g.size();
  if (n < 3) return g;
  bool all = col;
  for (int i = 2; i < n and all; ++i) {
    all &= orient(g[0], g[1], g[i]) == 0;
  }
  if (all) return g;
  vector<point> h;
  T lim = col ? 0 : 1;
  for (int i = 0; i <= 1; ++i) {
    int s = h.size();
    for (point p : g) {
      while (h.size() >= s + 2 and orient(h[h.size() - 2], h[h.size()
        - 1], p) < lim) {
        h.pop_back();
      }
      h.push_back(p);
    }
    h.pop_back();
    reverse(g.begin(), g.end());
  }
  return h;
}
```

ray

```
float ray_point(point a, point b, point p) {
  if (a != b) {
    line l(a, b);
    if (l.comp(a, p)) return l.dist(p);
  }
  return dist(p, a);
}

float ray_ray(point a, point b, point c, point d) {
  if (a == b) return ray_point(c, d, a);
  if (c == d) return ray_point(a, b, c);
  line l(a, b), r(c, d);
  if (cross(l.v, r.v) != 0) {
    point p = inter(l, r);
    if (l.comp(a, p) and r.comp(c, p)) return 0;
  }
  return min(ray_point(a, b, c), ray_point(c, d, a));
}
```

segment

```
bool in_disk(point a, point b, point p) {
    return dot(a - p, b - p) <= 0;
}

bool on_segment(point a, point b, point p) {
    return orient(a, b, p) == 0 and in_disk(a, b, p);
}

vector<point> proper_inter(point a, point b, point c, point d) {
    T oa = orient(c, d, a),
      ob = orient(c, d, b),
      oc = orient(a, b, c),
      od = orient(a, b, d);
    if (sign(oa) * sign(ob) >= 0 or sign(oc) * sign(od) >= 0) return
      {};
    return {(a * ob - b * oa) / (ob - oa)};
}

vector<point> inters(point a, point b, point c, point d) {
    vector<point> p = proper_inter(a, b, c, d);
    if (p.size()) return p;
    if (on_segment(c, d, a)) p.push_back(a);
    if (on_segment(c, d, b)) p.push_back(b);
    if (on_segment(a, b, c)) p.push_back(c);
    if (on_segment(a, b, d)) p.push_back(d);
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    return p;
}

float segment_point(point a, point b, point p) {
    if (a != b) {
        line l(a, b);
        if (l.comp(a, p) and l.comp(p, b)) return l.dist(p);
    }
    return min(dist(p, a), dist(p, b));
}

float segment_segment(point a, point b, point c, point d) {
    if (proper_inter(a, b, c, d).size()) return 0;
    return min({segment_point(a, b, c), segment_point(a, b, d),
                segment_point(c, d, a), segment_point(c, d, b)});
}
```

graph

binary-lifting

```
struct binary_lifting {
  int n, log;
  vector<int> d;
  vector<vector<int>> up;

  binary_lifting(vector<vector<int>> &g) {
    n = g.size();
    log = __bit_width(n);
    d.assign(n, -1);
    up.assign(log, vector<int>(n));
    for (int s = 0; s < n; ++s) if (d[s] == -1) {
      d[s] = 0;
      up[0][s] = s;
      queue<int> q;
      q.push(s);
      while (q.size()) {
        int a = q.front(); q.pop();
        for (int b : g[a]) if (d[b] == -1) {
          d[b] = d[a] + 1;
          up[0][b] = a;
          q.push(b);
        }
      }
    }
    for (int i = 1; i < log; ++i) {
      for (int a = 0; a < n; ++a) {
        up[i][a] = up[i - 1][up[i - 1][a]];
      }
    }
  }

  int kth(int a, int k) {
    k = min(k, d[a]);
    for (int i = 0; i < log; ++i) {
      if ((k >> i) & 1) a = up[i][a];
    }
    return a;
  }

  int lca(int a, int b) {
    if (d[a] < d[b]) swap(a, b);
    a = kth(a, d[a] - d[b]);
    if (a == b) return a;
    for (int i = log - 1; i >= 0; --i) {
      if (up[i][a] != up[i][b]) {
        a = up[i][a];
        b = up[i][b];
      }
    }
    return up[0][a];
  }

  int dist(int a, int b) {
    return d[a] + d[b] - 2 * d[lca(a, b)];
  }

  int path(int a, int b, int k) {
    int p = lca(a, b);
    int l = d[a] - d[p];
    int r = d[b] - d[p];
    k = min(k, l + r);
    if (k <= l) return kth(a, k);
    else return kth(b, l + r - k);
  }
};
```

centroid-decomposition

```
pair<vector<int>, vector<vector<int>>> centroid_decomposition(int r,
  vector<vector<int>> &g) {
  int n = g.size();
  vector<int> parent(n);
  vector<int> size(n);
  vector<bool> removed(n);
  vector<vector<int>> dist(n);
  auto dfs = [&](auto &self, int a, int p) -> int {
    size[a] = 1;
    for (int b : g[a]) if (b != p and not removed[b]) {
      size[a] += self(self, b, a);
    }
    return size[a];
  };
  auto centroid = [&](auto &self, int a, int p, int s) -> int {
    for (int b : g[a]) if ((b != p and not removed[b]) and size[b] >
      s / 2) {
      return self(self, b, a, s);
    }
    return a;
  };
  auto distance = [&](auto &self, int a, int p, int d) -> void {
    dist[a].push_back(d);
    for (int b : g[a]) if (b != p and not removed[b]) {
      self(self, b, a, d + 1);
    }
  };
  auto build = [&](auto &self, int a, int p) -> void {
    int s = dfs(dfs, a, a);
    a = centroid(centroid, a, a, s);
    distance(distance, a, a, 0);
    parent[a] = p;
    removed[a] = true;
    for (int b : g[a]) if (not removed[b]) {
      self(self, b, a);
```

```
    }
  };
  build(build, r, -1);
  for (auto &path : dist) reverse(path.begin(), path.end());
  return {parent, dist};
}
```

articulations

conectivity/

```
vector<int> articulations(int one, vector<vector<int>> &g) {
  int n = g.size(), timer = 1;
  vector<int> in(n), low(n);
  vector<bool> art(n);

  auto dfs = [&](auto &self, int a, int p) -> void {
    in[a] = low[a] = timer++;
    int c = 0;
    for (int b : g[a]) if (b != p) {
      if (in[b] != 0) {
        low[a] = min(low[a], in[b]);
      } else {
        c++;
        self(self, b, a);
        low[a] = min(low[a], low[b]);
        if (a != p and low[b] >= in[a]) art[a] = true;
      }
    }
    if (a == p and c > 1) art[a] = true;
  };

  vector<int> points;
  for (int i = one; i < n; ++i) {
    if (in[i] == 0) dfs(dfs, i, i);
    if (art[i]) points.push_back(i);
  }
  return points;
}
```

block-cut-tree

conectivity/

```
// Block-Cut Tree: decomposes an undirected graph into biconnected
  components
// Time: O(V + E) | Space: O(V + E)
// Note: returns a tree where nodes [0, N) are original vertices and
  nodes >= N are blocks
vector<vector<int>> block_cut_tree(vector<vector<int>> &g) {
  int n = g.size(), t = 0;
  vector<int> tin(n, -1), low(n), st;
  vector<vector<int>> bct(n); bct.reserve(2 * n);
  auto dfs = [&](auto &self, int a, int p) -> void {
    tin[a] = low[a] = t++;
    st.push_back(a);
    bool parent = true;
    for (int b : g[a]) {
      if (p == b and parent) {
        parent = false;
        continue;
      }
      if (tin[b] != -1) {
        low[a] = min(low[a], tin[b]);
      } else {
        self(self, b, a);
        low[a] = min(low[a], low[b]);
        if (tin[a] <= low[b]) {
          int i = bct.size();
          bct.emplace_back();
          bct[i].push_back(a);
          bct[a].push_back(i);
          while (true) {
            int c = st.back(); st.pop_back();
            bct[i].push_back(c);
            bct[c].push_back(i);
            if (b == c) break;
          }
        }
      }
    }
  };
  for (int i = 0; i < n; ++i) {
    if (tin[i] == -1) {
      dfs(dfs, i, -1);
    }
  }
  return bct;
}
```

bridge-tree

conectivity/

```
// Bridge Tree: condenses 2-edge-connected components into single
  nodes
// Time: O(V + E) | Space: O(V + E)
// Note: returns a tree and an id array where id[a] is the new node
  id of original vertex a
pair<vector<vector<int>>, vector<int>>
  bridge_tree(vector<vector<int>> &g) {
  int n = g.size(), t = 0;
  vector<int> tin(n, -1), low(n), id(n), st;
  vector<vector<int>> bt;
  auto dfs = [&](auto &self, int a, int p) -> void {
    tin[a] = low[a] = t++;
    st.push_back(a);
    bool parent = true;
    for (int b : g[a]) {
      if (b == p and parent) {
        parent = false;
        continue;
      }
      if (tin[b] != -1) {
        low[a] = min(low[a], tin[b]);
      } else {
        self(self, b, a);
        low[a] = min(low[a], low[b]);
      }
    }
    if (low[a] == tin[a]) {
      int i = bt.size();
      bt.emplace_back();
      while (true) {
        int c = st.back(); st.pop_back();
        id[c] = i;
        if (c == a) break;
      }
    }
  };
  for (int i = 0; i < n; ++i) {
    if (tin[i] == -1) {
      dfs(dfs, i, -1);
    }
  }
  for (int a = 0; a < n; ++a) {
    for (int b : g[a]) {
      if (id[a] != id[b]) {
        bt[id[a]].push_back(id[b]);
      }
    }
  }
  return {bt, id};
}
```

bridges

conectivity/

```
vector<pair<int, int>> bridges(int one, vector<vector<int>> &g) {
  int n = g.size(), timer = 1;
  vector<int> in(n), low(n);
  vector<pair<int, int>> e;

  auto dfs = [&](auto &self, int a, int p) -> void {
    in[a] = low[a] = timer++;
    for (int b : g[a]) if (b != p) {
      if (in[b] != 0) {
        low[a] = min(low[a], in[b]);
      } else {
        self(self, b, a);
        low[a] = min(low[a], low[b]);
        if (in[a] < low[b]) e.emplace_back(a, b);
      }
    }
  };

  for (int i = one; i < n; ++i) if (in[i] == 0) dfs(dfs, i, i);
  return e;
}
```

cycle

conectivity/

```
vector<int> find_cycle(bool one, bool is_directed,
  vector<vector<int>> &g) {
  int n = g.size();
  vector<int> state(n), parent(n);
  vector<int> cycle;
  auto dfs = [&](auto &self, int a, int p) -> bool {
    state[a] = 1;
    for (int b : g[a]) if (b != p or is_directed) {
      if (state[b] == 0) {
        parent[b] = a;
        if (self(self, b, a)) return true;
      } else if (state[b] == 1) {
        int c = a;
        while (c != b) {
          cycle.push_back(c);
          c = parent[c];
        }
        cycle.push_back(b);
        reverse(cycle.begin(), cycle.end());
        return true;
      }
    }
    state[a] = 2;
    return false;
  };
  for (int i = one; i < n; ++i) if (state[i] == 0) {
    if (dfs(dfs, i, -1)) return cycle;
  }
  return {};
}
```

dominator-tree

conectivity/

```
// Dominator Tree: computes immediate dominators in directed graphs
// Time: build O(V log V + E) | Space: O(V + E)
// Note: returns the tree where node a immediately dominates its
  children
struct dominator_dsu {
  vector<int> p, c;
  vector<int> &x;

  dominator_dsu(int n, vector<int> &x): p(n), c(n), x(x) {
    iota(p.begin(), p.end(), 0);
    iota(c.begin(), c.end(), 0);
  }

  int find(int a) {
    if (p[a] == a) return c[a];
    int r = p[a];
    if (p[r] != r) {
      find(r);
      if (x[c[r]] < x[c[a]]) c[a] = c[r];
      p[a] = p[r];
    }
    return c[a];
  }

  void unite(int a, int b) {
    p[b] = a;
  }
};

vector<vector<int>> dominator_tree(int s, vector<vector<int>> &g) {
  int n = g.size();
  vector<int> tin(n, -1), inv(n), parent(n);
  int t = 0;
  auto dfs = [&](auto &self, int a) -> void {
    tin[a] = t;
    inv[t] = a;
    t++;
    for (int b : g[a]) if (tin[b] == -1) {
      parent[b] = a;
      self(self, b);
    }
  };
  dfs(dfs, s);
  vector<vector<int>> rev(n);
  for (int a = 0; a < n; ++a) {
    for (int b : g[a]) rev[b].push_back(a);
  }
  vector<int> semi(n), dom(n);
  for (int a = 0; a < n; ++a) semi[a] = tin[a];
  vector<vector<int>> bucket(n);
  dominator_dsu dsu(n, semi);
  for (int i = t - 1; i >= 1; --i) {
    int a = inv[i];
    for (int b : rev[a]) if (tin[b] != -1) {
      semi[a] = min(semi[a], semi[dsu.find(b)]);
    }
    bucket[inv[semi[a]]].push_back(a);
    int p = parent[a];
    dsu.unite(p, a);
    for (int b : bucket[p]) {
      dom[b] = p;
      int c = dsu.find(b);
      if (semi[c] < semi[b]) dom[b] = c;
    }
    bucket[p].clear();
  }
  vector<vector<int>> dt(n);
  for (int i = 1; i < t; ++i) {
    int a = inv[i];
    if (dom[a] != inv[semi[a]]) dom[a] = dom[dom[a]];
    dt[dom[a]].push_back(a);
  }
  return dt;
}
```

steiner-tree

conectivity/

```
// Steiner Tree: minimum cost to connect k terminal nodes in a graph
  of n nodes
// Time: O(3^k * n + 2^k * m log n) | Space: O(2^k * n)
// Note: assumes terminals are nodes 0 to k-1 | g is an adjacency
  list (node, weight)
// State: steiner[(1<<k)-1][i] = min cost to connect all k terminals
```

```
  and node i
vector<vector<int>> steiner_tree(int k, const vector<vector<pair<int,
  int>>> &g) {
  const int n = g.size();
  vector<vector<int>> steiner(1 << k, vector<int>(n, inf));
  for (int i = 0; i < n; ++i) steiner[0][i] = 0;
  for (int i = 0; i < k; ++i) steiner[1 << i][i] = 0;
  for (int mask = 1; mask < (1 << k); ++mask) {
    for (int sub = mask; sub > 0; sub = mask & (sub - 1)) {
      for (int i = 0; i < n; ++i) {
        steiner[mask][i] = min(steiner[mask][i], steiner[sub][i] +
          steiner[mask ^ sub][i]);
      }
    }
    priority_queue<pair<int, int>, vector<pair<int, int>>,
      greater<pair<int, int>>> pq;
    for (int i = 0; i < n; ++i) if (steiner[mask][i] != inf) {
      pq.push({steiner[mask][i], i});
    }
    while (pq.size()) {
      auto [d, i] = pq.top(); pq.pop();
      if (steiner[mask][i] < d) continue;
      for (auto [j, w] : g[i]) if (steiner[mask][i] + w <
        steiner[mask][j]) {
        steiner[mask][j] = steiner[mask][i] + w;
        pq.push({steiner[mask][j], j});
      }
    }
  }
  return steiner;
}

// Steiner Tree: minimum cost to connect k terminal nodes in a graph
  of n nodes
// Time: O(3^k * n + 2^k * n^2) | Space: O(2^k * n)
// Note: assumes terminals are nodes 0 to k-1 | dist[u][v] must be
  the shortest path between u and v
// State: steiner[(1<<k)-1][i] = min cost to connect all k terminals
  and node i
vector<vector<int>> steiner_tree(int k, const vector<vector<int>>
  &dist) {
  const int n = dist.size();
  vector<vector<int>> steiner(1 << k, vector<int>(n, inf));
  for (int i = 0; i < n; ++i) steiner[0][i] = 0;
  for (int i = 0; i < k; ++i) steiner[1 << i][i] = 0;
  for (int mask = 1; mask < (1 << k); ++mask) {
    for (int sub = mask; sub > 0; sub = mask & (sub - 1)) {
      for (int i = 0; i < n; ++i) {
        steiner[mask][i] = min(steiner[mask][i], steiner[sub][i] +
          steiner[mask ^ sub][i]);
      }
    }
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        steiner[mask][j] = min(steiner[mask][j], steiner[mask][i] +
          dist[i][j]);
      }
    }
  }
  return steiner;
}
```

tarjan

conectivity/

```
vector<vector<int>> tarjan(int r, vector<vector<int>> &g) {
  int n = g.size(), timer = 1;
  vector<vector<int>> components;
  vector<int> id(n), low(n);
  stack<int> trail;
  vector<bool> in(n);

  auto dfs = [&](auto &self, int a) -> void {
    id[a] = low[a] = timer++;
    trail.push(a);
    in[a] = true;
    for (int b : g[a]) {
      if (id[b] == 0) {
        self(self, b);
        low[a] = min(low[a], low[b]);
      } else if (in[b]) {
        low[a] = min(low[a], id[b]);
      }
    }

    if (id[a] == low[a]) {
      vector<int> component;
      while (true) {
        int b = trail.top(); trail.pop();
        in[b] = false;
        component.push_back(b);
        if (b == a) break;
      }
      components.push_back(component);
    }
  };

  for (int a = r; a < n; ++a) {
    if (id[a] == 0) {
      dfs(dfs, a);
    }
  }

  reverse(components.begin(), components.end());
  return components;
}

pair<vector<vector<int>>, vector<int>> condensed_graph(int r,
  vector<vector<int>> &g) {
  int n = g.size();
  vector<vector<int>> components = tarjan(r, g);
  int m = components.size();

  vector<int> id(n);
  for (int i = 0; i < m; ++i) {
    for (int a : components[i]) {
      id[a] = i;
    }
  }

  vector<vector<int>> c(m);
  for (int a = r; a < n; ++a) {
    for (int b : g[a]) if (id[a] != id[b]) {
      c[id[a]].push_back(id[b]);
    }
  }

  for (int i = 0; i < m; ++i) {
    sort(c[i].begin(), c[i].end());
    c[i].erase(unique(c[i].begin(), c[i].end()), c[i].end());
  }
  return {c, id};
}
```

bellman-ford

distance/

```
vector<int> bellman_ford(int s, int n, vector<array<int, 3>> &e) {
  vector<int> d(n, inf);
  d[s] = 0;
  for (int i = 1; i < n; ++i) {
    bool chd = false;
    for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b]) {
      d[b] = d[a] + w;
      chd = true;
    }
    if (not chd) break;
  }
  for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b])
    return {};
  return d;
}

vector<int> spfa(int s, vector<vector<pair<int, int>>> &g) {
  int n = g.size();
  vector<int> d(n, inf), c(n);
  vector<bool> in(n);
  queue<int> q;
  d[s] = 0;
  in[s] = true;
  q.push(s);
  while (q.size()) {
    int a = q.front(); q.pop();
    in[a] = false;
    for (auto [b, w] : g[a]) if (d[a] + w < d[b]) {
      d[b] = d[a] + w;
      if (not in[b]) {
        c[b] += 1;
        if (c[b] >= n) return {};
        in[b] = true;
        q.push(b);
      }
    }
  }
  return d;
}
```

floyd-warshall

distance/

```
vector<vector<int>> floyd_warshall(int n, vector<vector<int>> &g) {
  vector<vector<int>> d = g;
  for (int i = 0; i < n; ++i) {
    d[i][i] = 0;
  }
  for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) if (d[i][k] != inf) {
      for (int j = 0; j < n; ++j) if (d[k][j] != inf) {
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
      }
    }
  }
  return d;
}
```

johnson

distance/

```
vector<vector<int>> johnson(vector<vector<pair<int, int>>> &g) {
  int n = g.size();
  g.emplace_back();
  for (int i = 0; i < n; ++i) {
    g[n].emplace_back(i, 0);
  }
  vector<int> h = spfa(n, g);
  g.pop_back();
  if (h.empty()) return {};
  h.pop_back();
  for (int i = 0; i < n; ++i) {
    for (auto &[j, w] : g[i]) {
      w += h[i] - h[j];
    }
  }
  vector<vector<int>> d;
  for (int i = 0; i < n; ++i) {
    d.push_back(dijkstra(i, g));
    for (int j = 0; j < n; ++j) {
      if (d[i][j] != inf) d[i][j] -= h[i] - h[j];
    }
  }
  return d;
}
```

dinic

flow/

```
struct dinic {
  struct edge {
    int b, w, i;
    bool h;
  };

  int n;
  vector<vector<edge>> g;
  vector<int> l, p;

  dinic(int n): n(n), g(n), l(n), p(n) {}

  void insert_directed(int a, int b, int w) {
    int i = g[b].size();
    int j = g[a].size();
    g[a].push_back({b, w, i, true});
    g[b].push_back({a, 0, j, false});
  }

  void insert_undirected(int a, int b, int w) {
    int i = g[b].size();
    int j = g[a].size();
    g[a].push_back({b, w, i, true});
    g[b].push_back({a, w, j, true});
  }

  bool bfs(int s, int t) {
    fill(l.begin(), l.end(), -1);
    l[s] = 0;
    queue<int> q;
    q.push(s);

    while (not q.empty()) {
      int a = q.front(); q.pop();
      for (auto &[b, w, i, h] : g[a]) {
        if (w > 0 and l[b] < 0) {
          l[b] = l[a] + 1;
          q.push(b);
        }
      }
    }
    return l[t] >= 0;
  }

  int dfs(int a, int t, int f) {
    if (a == t or f == 0) return f;

    for (int &k = p[a]; k < g[a].size(); ++k) {
      auto &[b, w, i, h] = g[a][k];
      if (w > 0 and l[b] == l[a] + 1) {
        if (int d = dfs(b, t, min(f, w))) {
          w -= d;
          g[b][i].w += d;
          return d;
        }
      }
    }
```

```
    return 0;
  }

  int flow(int s, int t) {
    int f = 0;
    while (bfs(s, t)) {
      fill(p.begin(), p.end(), 0);
      while (int d = dfs(s, t, inf)) {
        f += d;
      }
    }
    return f;
  }

  bool on_cut(int a) {
    return l[a] >= 0;
  }

  vector<pair<int, int>> saturated_edges() {
    vector<pair<int, int>> e;
    for (int a = 0; a < n; ++a) {
      for (auto [b, w, i, h] : g[a]) if (h and w == 0) {
        e.push_back({a, b});
      }
    }
    return e;
  }
};
```

min-cost-k-flow

flow/

```
// Min Cost K-Flow: successive shortest path with potentials
// Time: O(F * E log V) where F is flow | Space: O(V + E)
// Note: // Note: SPFA handles initial negative edge costs. If all
  costs are non-negative, replace spfa(s) with fill(h.begin(),
  h.end(), 0)
struct min_cost_k_flow {
  struct edge {
    int b, i, w, c;
    bool o;
  };

  int n;
  vector<vector<edge>> g;
  vector<int> d, h, p, e;

  min_cost_k_flow(int n): n(n), g(n), d(n), h(n), p(n), e(n) {}

  void insert_edge(int a, int b, int w, int c) {
    int i = g[b].size();
    int j = g[a].size();
    g[a].push_back({b, i, w, c, true});
    g[b].push_back({a, j, 0, -c, false});
  }

  bool spfa(int s) {
    fill(h.begin(), h.end(), inf);
    vector<int> cnt(n);
    vector<bool> in(n);
    queue<int> q;
    h[s] = 0;
    in[s] = true;
    q.push(s);
    while (q.size()) {
      int a = q.front(); q.pop();
      in[a] = false;
      for (auto &[b, i, w, c, o] : g[a]) if (w > 0 and h[a] + c <
        h[b]) {
        h[b] = h[a] + c;
        if (not in[b]) {
          cnt[b] += 1;
          if (cnt[b] >= n) return false;
          in[b] = true;
          q.push(b);
        }
      }
    }
    return true;
  }

  bool dijkstra(int s, int t) {
    fill(d.begin(), d.end(), inf);
    priority_queue<pair<int, int>, vector<pair<int, int>>,
      greater<pair<int, int>>> pq;
    d[s] = 0;
    pq.emplace(0, s);
    while (pq.size()) {
      auto [x, a] = pq.top(); pq.pop();
      if (d[a] < x) continue;

      for (int i = 0; i < g[a].size(); ++i) {
        auto &[b, j, w, c, o] = g[a][i];
        if (w <= 0) continue;

        int v = d[a] + c + h[a] - h[b];
        if (v < d[b]) {
          d[b] = v;
          p[b] = a;
          e[b] = i;
          pq.emplace(d[b], b);
        }
      }
    }
    return d[t] != inf;
  }

  pair<int, int> cost_flow(int s, int t, int k = inf) {
    int flow = 0, cost = 0;
    if (not spfa(s)) return {-1, -1};
    while (flow < k and dijkstra(s, t)) {
      for (int a = 0; a < n; ++a) if (d[a] != inf) {
        h[a] += d[a];
      }
      int push = k - flow;
      for (int a = t; a != s; a = p[a]) {
        int b = p[a], i = e[a];
        push = min(push, g[b][i].w);
      }
      for (int a = t; a != s; a = p[a]) {
        int b = p[a], i = e[a], j = g[b][i].i;
        g[b][i].w -= push;
        g[a][j].w += push;
        cost += g[b][i].c * push;
      }
      flow += push;
    }
    return {flow, cost};
  }

  vector<pair<int, int>> saturated_edges() {
    vector<pair<int, int>> edges;
    for (int a = 0; a < n; ++a) {
      for (auto &[b, i, w, c, o] : g[a]) if (w == 0 and o) {
        edges.push_back({a, b});
      }
    }
    return edges;
  }
};
```

heavy-light-decomposition

```
// Heavy-Light Decomposition: splits a path into O(log N) ranges of
  positions for an external structure
// Time: O(N) build, O(log N) per path | Space: O(N)
// Usage: hld h(g, false) values on vertices, hld h(g, true) values
  on edges | vertex v is at h.id[v], edge u-v at h.edge(u, v)
struct hld {
  int t;
  bool e;
  vector<int> p, h, s, id;

  hld(vector<vector<int>> &g, bool e): t(0), e(e) {
    int n = g.size();
    p.resize(n);
    h.resize(n);
    s.resize(n);
    id.resize(n);
    for (int i = 0; i < n; ++i) if (s[i] == 0) {
      p[i] = h[i] = i;
      dfs(i, g);
      decompose(i, g);
    }
  }

  void dfs(int a, vector<vector<int>> &g) {
    s[a] = 1;
    for (int &b : g[a]) if (b != p[a]) {
      p[b] = a;
      dfs(b, g);
      s[a] += s[b];
      if (g[a][0] == p[a] or s[b] > s[g[a][0]]) {
        swap(b, g[a][0]);
      }
    }
  }

  void decompose(int a, vector<vector<int>> &g) {
    id[a] = t++;
    for (int b : g[a]) if (b != p[a]) {
      h[b] = (b == g[a][0] ? h[a] : b);
      decompose(b, g);
    }
  }

  int edge(int u, int v) {
    return max(id[u], id[v]);
  }

  vector<tuple<int, int, bool>> path(int a, int b) {
    vector<tuple<int, int, bool>> up, down;
    while (h[a] != h[b]) {
      if (id[h[a]] > id[h[b]]) {
        up.push_back({id[h[a]], id[a], true});
        a = p[h[a]];
      } else {
        down.push_back({id[h[b]], id[b], false});
        b = p[h[b]];
      }
    }
    if (id[a] > id[b]) {
      up.push_back({id[b] + e, id[a], true});
    } else if (id[a] + e <= id[b]) {
      down.push_back({id[a] + e, id[b], false});
    }
    up.insert(up.end(), down.rbegin(), down.rend());
    return up;
  }

  tuple<int, int> subtree(int v) {
    return {id[v] + e, id[v] + s[v] - 1};
  }
};
```

hierholzer

```
// Undirected Eulerian Path: Hierholzer's algorithm for paths and
  circuits
// Time: O(V + E) | Space: O(V + E)
// Note: returns sequence of vertices or empty if impossible | s must
  be a valid start node
vector<int> undirected_eulerian_path(int s, vector<vector<pair<int,
  int>>> &g, int e) {
  int n = g.size();
  int odd = 0;
  for (int i = 0; i < n; ++i) {
    if (g[i].size() & 1) odd++;
  }
  if (odd != 0 and odd != 2) return {};
  if (odd == 2 and (g[s].size() & 1) == 0) return {};
  vector<bool> used(e + 1);
  vector<int> next(n), path;
  auto hierholzer = [&](auto &self, int a) -> void {
    while (next[a] < g[a].size()) {
      auto [b, id] = g[a][next[a]];
      next[a]++;
      if (not used[id]) {
        used[id] = true;
        self(self, b);
      }
    }
    path.push_back(a);
  };
  hierholzer(hierholzer, s);
  if (path.size() != e + 1) return {};
  reverse(path.begin(), path.end());
  return path;
}

// Directed Eulerian Path: Hierholzer's algorithm for paths and
  circuits
// Time: O(V + E) | Space: O(V + E)
// Note: returns sequence of vertices or empty if impossible | s must
  be a valid start node
vector<int> directed_eulerian_path(int s, vector<vector<int>> &g, int
  e) {
  int n = g.size();
  vector<int> in(n), out(n);
  for (int a = 0; a < n; ++a) {
    out[a] = g[a].size();
    for (int b : g[a]) {
      in[b]++;
    }
  }
  int start = 0, end = 0;
  for (int i = 0; i < n; ++i) {
    if (out[i] - in[i] == +1) start++;
    else if (out[i] - in[i] == -1) end++;
    else if (out[i] - in[i] != 0) return {};
  }
  if (not ((start == 0 and end == 0) or (start == 1 and end == 1)))
    return {};
  if (start == 1 and out[s] - in[s] != +1) return {};
  vector<int> next(n), path;
  auto hierholzer = [&](auto &self, int a) -> void {
    while (next[a] < g[a].size()) {
```

```
      int b = g[a][next[a]];
      next[a]++;
      self(self, b);
    }
    path.push_back(a);
  };
  hierholzer(hierholzer, s);
  if (path.size() != e + 1) return {};
  reverse(path.begin(), path.end());
  return path;
}
```

leaf-peeling

```
// Leaf Peeling: finds the center(s) and diameter of an undirected
  tree
// Time: O(N) | Space: O(N)
// Note: returns {diameter, centers} | 'i' sets 0-based or 1-based
  indexing
pair<int, vector<int>> leaf_peeling(int i, vector<vector<int>> &g) {
  int n = g.size() - i;
  if (n == 1) return {0, {i}};
  vector<int> d(g.size());
  queue<int> q;
  for (int a = i; a < g.size(); ++a) {
    d[a] = g[a].size();
    if (d[a] == 1) q.push(a);
  }
  int r = n, l = 0;
  while (r > 2) {
    int s = q.size();
    r -= s;
    l++;
    while (s--) {
      int a = q.front(); q.pop();
      for (int b : g[a]) {
        if (--d[b] == 1) q.push(b);
      }
    }
  }
  vector<int> c;
  while (q.size()) c.push_back(q.front()), q.pop();
  return {2 * l + (c.size() == 2), c};
}
```

link-cut-tree

```
// Link-Cut Tree: dynamic forest with link, cut, connected, lca, path
  query, path update and set
// Time: O(log N) amortized per operation | Space: O(N)
// Note: vertices are 1-based (a[0] is ignored) | lca is 0 if not
  connected | edit node, pull and apply
struct link_cut_tree {
  struct node {
    int val = 0, size = 0;
    int sum = 0, lazy = 0;
    bool rev = false;
    int p = 0;
    array<int, 2> c = {0, 0};
  };

  void pull(int n) {
    node &a = tree[n], &l = tree[a.c[0]], &r = tree[a.c[1]];
    a.size = l.size + 1 + r.size;
    a.sum = l.sum + a.val + r.sum;
  }

  void apply(int n, int x) {
    if (n == 0) return;
    tree[n].val += x;
    tree[n].sum += x * tree[n].size;
    tree[n].lazy += x;
  }

  vector<node> tree;

  link_cut_tree(const vector<int> &a): tree(a.size()) {
    for (int i = 1; i < a.size(); ++i) {
      tree[i].val = a[i];
      pull(i);
    }
  }

  void flip(int n) {
    if (n == 0) return;
    swap(tree[n].c[0], tree[n].c[1]);
    tree[n].rev ^= 1;
  }

  void push(int n) {
    if (tree[n].rev) flip(tree[n].c[0]), flip(tree[n].c[1]),
      tree[n].rev = false;
    if (tree[n].lazy != tree[0].lazy) {
      apply(tree[n].c[0], tree[n].lazy), apply(tree[n].c[1],
        tree[n].lazy);
      tree[n].lazy = tree[0].lazy;
    }
  }

  bool is_root(int n) {
    int p = tree[n].p;
    return tree[p].c[0] != n and tree[p].c[1] != n;
  }

  void rotate(int n) {
    int p = tree[n].p, g = tree[p].p, d = tree[p].c[1] == n;
    if (not is_root(p)) tree[g].c[tree[g].c[1] == p] = n;
    tree[n].p = g;
    tree[p].c[d] = tree[n].c[d ^ 1];
    if (tree[n].c[d ^ 1]) tree[tree[n].c[d ^ 1]].p = p;
    tree[n].c[d ^ 1] = p;
    tree[p].p = n;
    pull(p), pull(n);
  }

  void splay(int n) {
    static vector<int> s;
    s = {n};
    for (int a = n; not is_root(a); a = tree[a].p)
      s.push_back(tree[a].p);
    while (s.size()) push(s.back()), s.pop_back();
    while (not is_root(n)) {
      int p = tree[n].p, g = tree[p].p;
      if (not is_root(p)) rotate((tree[g].c[1] == p) == (tree[p].c[1]
        == n) ? p : n);
      rotate(n);
    }
  }

  int access(int a) {
    int last = 0;
    for (int x = a; x != 0; x = tree[x].p) {
      splay(x);
      tree[x].c[1] = last;
      pull(x);
      last = x;
    }
    splay(a);
    return last;
  }

  void make_root(int a) {
    access(a);
    flip(a);
  }

  int find_root(int a) {
    access(a);
    while (tree[a].c[0] != 0) push(a), a = tree[a].c[0];
    splay(a);
    return a;
  }

  void link(int a, int b) {
    make_root(a);
    if (find_root(b) != a) tree[a].p = b;
  }

  void cut(int a, int b) {
    make_root(a);
    access(b);
    if (tree[b].c[0] == a and tree[a].c[1] == 0) {
      tree[b].c[0] = tree[a].p = 0;
      pull(b);
    }
  }

  bool connected(int a, int b) {
    return find_root(a) == find_root(b);
  }

  int lca(int a, int b) {
    if (not connected(a, b)) return 0;
    access(a);
    return access(b);
  }

  void set(int a, int x) {
    access(a);
    tree[a].val = x;
    pull(a);
  }

  void update(int a, int b, int x) {
    make_root(a);
    access(b);
    apply(b, x);
  }

  node query(int a, int b) {
    make_root(a);
    access(b);
    return tree[b];
  }
};
```

hopcroft-karp

matching/

```
struct hopcroft_karp {
  const int inf = 1e9;
  int n, m, s;
  vector<int> r, l, d;
  vector<vector<int>> g;

  hopcroft_karp(int n, int m): n(n), m(m), r(n, -1), l(m, -1), d(n),
    g(n) {}

  void edge(int a, int b) {
    g[a].push_back(b);
  }

  bool bfs() {
    s = inf;
    queue<int> q;
    for (int a = 0; a < n; ++a) {
      if (r[a] == -1) {
        d[a] = 0;
        q.push(a);
      } else {
        d[a] = inf;
      }
    }
    while (q.size()) {
      int a = q.front(); q.pop();
      if (d[a] >= s) continue;
      for (int b : g[a]) {
        if (l[b] == -1) {
          s = min(s, d[a] + 1);
        } else if (d[l[b]] == inf) {
          d[l[b]] = d[a] + 1;
          q.push(l[b]);
        }
      }
    }
    return s != inf;
  }

  bool dfs(int a) {
    if (a == -1) return true;
    for (int b : g[a]) {
      int h = (l[b] != -1) ? d[l[b]] : s;
      if (h == d[a] + 1) if (dfs(l[b])) {
        r[a] = b;
        l[b] = a;
        return true;
      }
    }
    d[a] = inf;
    return false;
  }

  int match() {
    int matching = 0;
    while (bfs()) {
      for (int a = 0; a < n; ++a) if (r[a] == -1 and dfs(a))
        matching++;
    }
    return matching;
  }

  vector<pair<int, int>> matches() {
    vector<pair<int, int>> m;
    for (int a = 0; a < n; ++a) if (r[a] != -1) m.emplace_back(a,
      r[a]);
    return m;
  }
};
```

hungarian

matching/

```
// Hungarian Algorithm: minimum weight bipartite matching
// Time: O(N^2 * M) | Space: O(N + M)
// Note: computes min cost assignment | negate input matrix for max
pair<int, vector<int>> hungarian(vector<vector<int>> g) {
  int n = g.size(), m = g[0].size();
  bool transposed = false;
  if (n > m) {
```

```
    transposed = true;
    vector<vector<int>> t(m, vector<int>(n));
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < m; ++j) {
        t[j][i] = g[i][j];
      }
    }
    swap(n, m);
    swap(g, t);
  }

  vector<int> pot_row(n + 1), pot_col(m + 1);
  vector<int> match(m + 1), prev_col(m + 1);

  for (int i = 1; i <= n; ++i) {
    match[0] = i;
    int col = 0;
    vector<int> min_cost(m + 1, inf);
    vector<bool> v(m + 1);
    do {
      v[col] = true;
      int row = match[col];
      int delta = inf;
      int next_col = 0;
      for (int j = 1; j <= m; ++j) if (not v[j]) {
        int cost = g[row - 1][j - 1] - pot_row[row] - pot_col[j];
        if (cost < min_cost[j]) {
          min_cost[j] = cost;
          prev_col[j] = col;
        }
        if (min_cost[j] < delta) {
          delta = min_cost[j];
          next_col = j;
        }
      }
      for (int j = 0; j <= m; ++j) {
        if (v[j]) {
          pot_row[match[j]] += delta;
          pot_col[j] -= delta;
        } else {
          min_cost[j] -= delta;
        }
      }
      col = next_col;
    } while (match[col] != 0);
    do {
      int prev = prev_col[col];
      match[col] = match[prev];
      col = prev;
    } while (col != 0);
  }

  int cost = -pot_col[0];
  vector<int> answer(n, -1);
  for (int j = 1; j <= m; ++j) if (match[j] != 0) {
    answer[match[j] - 1] = j - 1;
  }

  if (transposed) {
    vector<int> real(m, -1);
    for (int i = 0; i < n; ++i) if (answer[i] != -1) {
      real[answer[i]] = i;
    }
    swap(answer, real);
  }
  return {cost, answer};
}
```

kuhn

matching/

```
// Kuhn's Algorithm: maximum bipartite matching
// Time: O(V * E) | Space: O(N + M)
// Note: connects left [0..n] to right [0..m] | match[right] returns
  left node or -1
vector<int> kuhn(int n, int m, vector<vector<int>> &g) {
  vector<int> match(m + 1, -1);
  int itr = 1;
  vector<int> v(n + 1);

  auto dfs = [&](auto &self, int a) -> bool {
    if (v[a] == itr) return false;
    v[a] = itr;
    for (int b : g[a]) if (match[b] == -1 or self(self, match[b])) {
      match[b] = a;
      return true;
    }
    return false;
  };

  for (int a = 0; a <= n; ++a, ++itr) {
    dfs(dfs, a);
  }
  return match;
}
```

maximum-clique

```
vector<int> maximum_clique(vector<vector<int>> &g) {
  int n = g.size();
  vector<int> h(n);
  for (int a = 0; a < n; ++a) {
    for (int b : g[a]) {
      h[a] |= 1LL << b;
    }
  }
  int size = 0, mask = 0;
  auto bron_kerbosch = [&](auto &self, int r, int p, int x) -> void {
    if (p == 0 and x == 0) {
      int s = __builtin_popcountll(r);
      if (s > size) {
        size = s;
        mask = r;
      }
      return;
    }
    int i = __builtin_ctzll(p | x);
    int c = p & ~h[i];
    while (c) {
      int k = __builtin_ctzll(c);
      int k_mask = 1LL << k;
      self(self, r | k_mask, p & h[k], x & h[k]);
      p ^= k_mask;
      x |= k_mask;
      c ^= k_mask;
    }
  };
  int initial = (1LL << n) - 1;
  bron_kerbosch(bron_kerbosch, 0, initial, 0);
  vector<int> clique;
  for (int i = 0; i < n; ++i) {
    if ((mask >> i) & 1) {
      clique.push_back(i);
    }
  }
  return clique;
}
```

two-sat

```
// 2-SAT: Solves Boolean Satisfiability for 2-CNF Formulas
// Time: Build O(N + M) | Space: O(N + M)
// Note: Uses Tarjan's SCC | 0-indexed variables
struct two_sat {
  int n;
  vector<vector<int>> g;

  two_sat(int n): n(n), g(2 * n) {}

  // A or B
  void disjunction(int a, bool n_a, int b, bool n_b) {
    a = (2 * a) ^ n_a;
    b = (2 * b) ^ n_b;
    g[a ^ 1].push_back(b);
    g[b ^ 1].push_back(a);
  }

  // A and B
  void conjunction(int a, bool n_a, int b, bool n_b) {
    disjunction(a, n_a, a, n_a);
    disjunction(b, n_b, b, n_b);
  }

  // A -> B
  void implication(int a, bool n_a, int b, bool n_b) {
    disjunction(a, !n_a, b, n_b);
  }

  // A == B
  void equal(int a, bool n_a, int b, bool n_b) {
    implication(a, n_a, b, n_b);
    implication(b, n_b, a, n_a);
  }

  // A != B / A xor B
  void not_equal(int a, bool n_a, int b, bool n_b) {
    disjunction(a, n_a, b, n_b);
    disjunction(a, !n_a, b, !n_b);
  }

  // Solves the 2-SAT and returns a valid assignment, or empty if
    unsatisfiable
  vector<bool> build() {
    vector<vector<int>> components = tarjan(0, g);
    int m = components.size();
    vector<int> id(2 * n);
    for (int i = 0; i < m; ++i) {
      for (int a : components[i]) {
        id[a] = i;
      }
    }
    vector<bool> assignment(n);
    for (int i = 0; i < n; ++i) {
      if (id[2 * i] == id[(2 * i) ^ 1]) return {};
      assignment[i] = id[2 * i] > id[(2 * i) ^ 1];
    }
    return assignment;
  }
};
```

math

berlekamp-massey

```
// Berlekamp-Massey: finds the shortest linear recurrence of a
  sequence
// Time: O(N^2) where N is sequence size | Space: O(N)
// Note: requires at least 2L terms to correctly find a recurrence of
  length L
vector<int> berlekamp_massey(const vector<int> &seq) {
  vector<int> old = {1}, curr = {1};
  int prev = 1, len = 0, shift = 1;
  for (int i = 0; i < seq.size(); ++i) {
    int delta = 0;
    for (int j = 0; j <= len; ++j) {
      delta = (delta + curr[j] * seq[i - j]) % mod;
    }
    if (delta == 0) {
      shift++;
      continue;
    }
    vector<int> temp = curr;
    int mult = (mod - (delta * mod_pow(prev, mod - 2)) % mod) % mod;
    if (curr.size() <= old.size() + shift) {
      curr.resize(old.size() + shift + 1, 0);
    }
    for (int j = 0; j < old.size(); ++j) {
      curr[j + shift] = (curr[j + shift] + mult * old[j]) % mod;
    }
    if (2 * len <= i) {
      len = i + 1 - len;
      old = temp;
      prev = delta;
      shift = 1;
    } else {
      shift++;
    }
  }
  vector<int> coef(len);
  for (int i = 1; i <= len; ++i) {
    coef[i - 1] = (mod - curr[i]) % mod;
  }
  return coef;
}

// Linear Recurrence K-th Term: computes the K-th term of a sequence
// Time: O(L^2 log K) where L is recurrence length | Space: O(L)
// Note: 0-indexed | the sequence must have at least L terms
int kth_term(int k, const vector<int> &seq, const vector<int> &coef) {
  if (k < seq.size()) return seq[k];
  if (coef.empty()) return 0;

  int n = coef.size();

  auto combine = [&](const vector<int> &a, const vector<int> &b) ->
    vector<int> {
    vector<int> r(2 * n);
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        r[i + j] = (r[i + j] + a[i] * b[j]) % mod;
      }
    }
    for (int i = 2 * n - 2; i >= n; --i) {
      for (int j = 0; j < n; ++j) {
        r[i - j - 1] = (r[i - j - 1] + r[i] * coef[j]) % mod;
      }
    }
    r.resize(n);
    return r;
  };

  vector<int> pol(n), exp(n);
  pol[0] = 1;
  if (n > 1) exp[1] = 1;
  else exp[0] = coef[0];

  while (k != 0) {
    if (k & 1) pol = combine(pol, exp);
```

```
    exp = combine(exp, exp);
    k >>= 1;
  }

  int ans = 0;
  for (int i = 0; i < n; ++i) {
    ans = (ans + pol[i] * seq[i]) % mod;
  }
  return ans;
}
```

fast-fourier-transform

convolution/

```
const float pi = acos(-1);
vector<complex<float>> root, inv_root;

void roots(int n) {
  if (n <= 2 * root.size()) return;

  root.resize(n / 2);
  inv_root.resize(n / 2);

  root[0] = {1, 0};
  inv_root[0] = {1, 0};

  float ang = 2 * pi / n;
  int half = n / 2;
  for (int i = 1; i < half; i *= 2) {
    complex<float> step(cos(ang * i), sin(ang * i));
    for (int j = 0; j < i; ++j) {
      root[i + j] = root[j] * step;
    }
  }
  for (int i = 1; i < half; ++i) {
    inv_root[i] = conj(root[i]);
  }
}

void fft(vector<complex<float>> &p, bool inv) {
  int n = p.size();
  roots(n);
  if (not inv) {
    for (int len = n; len >= 2; len /= 2) {
      int half = len / 2;
      int step = 2 * root.size() / len;
      for (int i = 0; i < n; i += len) {
        for (int j = 0; j < half; ++j) {
          auto w = root[step * j];
          auto x = p[i + j];
          auto y = p[i + j + half];
          p[i + j] = x + y;
          p[i + j + half] = (x - y) * w;
        }
      }
    }
  } else {
    for (int len = 2; len <= n; len *= 2) {
      int half = len / 2;
      int step = 2 * root.size() / len;
      for (int i = 0; i < n; i += len) {
        for (int j = 0; j < half; ++j) {
          auto w = inv_root[step * j];
          auto x = p[i + j];
          auto y = p[i + j + half] * w;
          p[i + j] = x + y;
          p[i + j + half] = x - y;
        }
      }
    }
    for (auto &x : p) x /= n;
  }
}

vector<int> convolution(vector<int> &a, vector<int> &b) {
  int n = a.size() + b.size() - 1;
  int m = 1;
  while (m < n) m *= 2;

  vector<complex<float>> fa(a.begin(), a.end());
  vector<complex<float>> fb(b.begin(), b.end());
  fa.resize(m);
  fb.resize(m);
  fft(fa, false);
  fft(fb, false);

  for (int i = 0; i < m; ++i) fa[i] *= fb[i];
  fft(fa, true);

  vector<int> c(n);
  for (int i = 0; i < n; ++i) c[i] = round(fa[i].real());
  return c;
}
```

fast-walsh-hadamard-transform

convolution/

```
// FWHT / Bitwise Convolution: C[k] = sum(A[i] * B[j]) for i (OPR) j
  = k
// Time: O(N log N) | Space: O(N)
// Operation: 'A' (AND), 'O' (OR), 'X' (XOR)
void fwht(vector<int> &p, int sign, char opr) {
  int n = p.size();

  for (int half = 1; half < n; half *= 2) {
    for (int i = 0; i < n; i += 2 * half) {
      for (int j = 0; j < half; ++j) {
        int x = p[i + j];
        int y = p[(i + j) + half];
        if (opr == 'A') {
          p[i + j] = ((x + sign * y) + mod) % mod;
        } else if (opr == 'O') {
          p[(i + j) + half] = ((sign * x + y) + mod) % mod;
        } else if (opr == 'X') {
          p[i + j] = (x + y) % mod;
          p[(i + j) + half] = ((x - y) + mod) % mod;
        }
      }
    }
  }

  if (opr == 'X' and sign == -1) {
    int inv_n = mod_pow(n, mod - 2);
    for (int &i : p) i = (i * inv_n) % mod;
  }
}

vector<int> convolution(vector<int> a, vector<int> b, char opr) {
  int n = max(a.size(), b.size());
  int m = 1;
  while (m < n) m *= 2;

  a.resize(m);
  b.resize(m);
  fwht(a, +1, opr);
  fwht(b, +1, opr);

  for (int i = 0; i < m; ++i) a[i] = (a[i] * b[i]) % mod;
  fwht(a, -1, opr);

  return a;
}
```

modulo-transform-technique

convolution/

```
vector<int> roots(int half, int ang, int mod) {
  vector<int> w(half);
  w[0] = 1;
  for (int i = 1; i < half; ++i) w[i] = (ang * w[i - 1]) % mod;
  return w;
}

void ntt(vector<int> &p, bool inv, int g, int mod) {
  int n = p.size();
  if (not inv) {
    for (int len = n; len >= 2; len /= 2) {
      int half = len / 2;
      int ang = mod_pow(g, (mod - 1) / len, mod);
      vector<int> w = roots(half, ang, mod);
      for (int i = 0; i < n; i += len) {
        for (int j = 0; j < half; ++j) {
          int x = p[i + j];
          int y = p[i + j + half];
          p[i + j] = (x + y) % mod;
          p[i + j + half] = (((x - y + mod) % mod) * w[j]) % mod;
        }
      }
    }
  } else {
    for (int len = 2; len <= n; len *= 2) {
      int half = len / 2;
      int ang = mod_pow(g, (mod - 1) / len, mod);
      ang = mod_pow(ang, mod - 2, mod);
      vector<int> w = roots(half, ang, mod);
      for (int i = 0; i < n; i += len) {
        for (int j = 0; j < half; ++j) {
          int x = p[i + j];
          int y = (p[i + j + half] * w[j]) % mod;
          p[i + j] = (x + y) % mod;
          p[i + j + half] = (x - y + mod) % mod;
        }
      }
    }
    int inv_n = mod_pow(n, mod - 2, mod);
    for (int &x : p) x = (x * inv_n) % mod;
  }
}
```

```
vector<int> convolution(vector<int> a, vector<int> b, int g = 3, int
  mod = 998244353) {
  int n = a.size() + b.size() - 1;
  int m = 1;
  while (m < n) m *= 2;

  a.resize(m);
  b.resize(m);
  ntt(a, false, g, mod);
  ntt(b, false, g, mod);

  for (int i = 0; i < m; ++i) a[i] = (a[i] * b[i]) % mod;
  ntt(a, true, g, mod);
  a.resize(n);

  return a;
}

vector<int> mtt(vector<int> &a, vector<int> &b, int mod) {
  vector<int> mods = {998244353, 1004535809, 469762049};

  vector<int> c1 = convolution(a, b, 3, mods[0]);
  vector<int> c2 = convolution(a, b, 3, mods[1]);
  vector<int> c3 = convolution(a, b, 3, mods[2]);

  int n = c1.size();
  vector<int> c(n);
  for (int i = 0; i < n; ++i) {
    vector<int> rems = {c1[i], c2[i], c3[i]};
    int128 x = chinese_remainder_theorem(rems, mods);
    c[i] = x % mod;
  }
  return c;
}
```

number-theoretic-transform

convolution/

```
const int g = 3;
const int mod = 998244353;

vector<int> root, inv_root;

void roots(int n) {
  if (n <= 2 * root.size()) return;

  root.resize(n / 2);
  inv_root.resize( n / 2);

  root[0] = 1;
  inv_root[0] = 1;

  int ang = mod_pow(g, (mod - 1) / n);
  int inv_ang = mod_pow(ang, mod - 2);

  int half = n / 2;
  for (int i = 1; i < half; ++i) {
    root[i] = (ang * root[i - 1]) % mod;
    inv_root[i] = (inv_ang * inv_root[i - 1]) % mod;
  }
}

void ntt(vector<int> &p, bool inv) {
  int n = p.size();
  roots(n);
  if (not inv) {
    for (int len = n; len >= 2; len /= 2) {
      int half = len / 2;
      int step = 2 * root.size() / len;
      for (int i = 0; i < n; i += len) {
        for (int j = 0; j < half; ++j) {
          int w = root[step * j];
          int x = p[i + j];
          int y = p[i + j + half];
          p[i + j] = (x + y) % mod;
          p[i + j + half] = (((x - y + mod) % mod) * w) % mod;
        }
      }
    }
  } else {
    for (int len = 2; len <= n; len *= 2) {
      int half = len / 2;
      int step = 2 * root.size() / len;
      for (int i = 0; i < n; i += len) {
        for (int j = 0; j < half; ++j) {
          int w = inv_root[step * j];
          int x = p[i + j];
          int y = (p[i + j + half] * w) % mod;
          p[i + j] = (x + y) % mod;
          p[i + j + half] = (x - y + mod) % mod;
        }
      }
    }
    int n_inv = mod_pow(n, mod - 2);
    for (int &x : p) x = (x * n_inv) % mod;
  }
}
```

```
    int inv_n = mod_pow(n, mod - 2);
    for (int &x : p) x = (x * inv_n) % mod;
  }
}

vector<int> convolution(vector<int> a, vector<int> b) {
  int n = a.size() + b.size() - 1;
  int m = 1;
  while (m < n) m *= 2;

  a.resize(m);
  b.resize(m);
  ntt(a, false);
  ntt(b, false);

  for (int i = 0; i < m; ++i) a[i] = (a[i] * b[i]) % mod;
  ntt(a, true);
  a.resize(n);

  return a;
}
```

gaussian-elimination

```
// Gaussian Elimination: solves systems of linear equations (modulo
  arithmetic)
// Time: O(N * M^2) | Space: O(M) auxiliary
// Note: returns {(0: none, 1: unique, 2: infinite), solution}
pair<int, vector<int>> gaussian_elimination(matrix &a, const int mod)
  {
  int n = a.size(), m = a[0].size() - 1;
  vector<int> p(m, -1);
  int r, c;
  for (r = 0, c = 0; r < n and c < m; ++c) {
    int k = r;
    for (int i = r + 1; i < n and a[k][c] == 0; ++i) k = i;
    if (a[k][c] == 0) continue;

    swap(a[r], a[k]);
    p[r] = c;

    int inv = mod_pow(a[r][c], mod - 2, mod);
    for (int j = c; j <= m; ++j) {
      a[r][j] = (a[r][j] * inv) % mod;
    }

    for (int i = 0; i < n; ++i) if (i != r and a[i][c] != 0) {
      int f = mod - a[i][c];
      for (int j = c; j <= m; ++j) {
        a[i][j] = (a[i][j] + f * a[r][j]) % mod;
      }
    }
    ++r;
  }
  for (int i = r; i < n; ++i) if (a[i][m] != 0) {
    return {0, {}};
  }
  vector<int> x(m);
  for (int i = 0; i < r; ++i) {
    x[p[i]] = a[i][m];
  }
  return {(r == m) ? 1 : 2, x};
}

// Gaussian Elimination: solves systems of linear equations
  (floating-point with partial pivoting)
// Time: O(N * M^2) | Space: O(M) auxiliary
// Note: returns {(0: none, 1: unique, 2: infinite), solution}. Uses
  'eps' for numerical stability.
pair<int, vector<float>> gaussian_elimination(matrix &a, const float
  eps) {
  int n = a.size(), m = a[0].size() - 1;
  vector<int> p(m, -1);
  int r, c;
  for (r = 0, c = 0; r < n and c < m; ++c) {
    int k = r;
    for (int i = r + 1; i < n; ++i) if (abs(a[i][c]) > abs(a[k][c]))
      k = i;
    if (abs(a[k][c]) < eps) continue;

    swap(a[r], a[k]);
    p[r] = c;

    float d = a[r][c];
    for (int j = c; j <= m; ++j) {
      a[r][j] /= d;
    }

    for (int i = 0; i < n; ++i) if (i != r and abs(a[i][c]) > eps) {
      float f = a[i][c];
      for (int j = c; j <= m; ++j) {
        a[i][j] -= f * a[r][j];
      }
    }
    ++r;
  }
  for (int i = r; i < n; ++i) if (abs(a[i][m]) > eps) {
    return {0, {}};
  }
  vector<float> x(m);
  for (int i = r; i < n; ++i) {
    x[p[i]] = a[i][m];
  }
  return {(r == m) ? 1 : 2, x};
}

// XOR Gaussian Elimination: solves XOR systems (mod 2)
// Time: O(N * n * m / 64) | Space: O(N * m / 64) auxiliary
// Note: returns {(-1: none, r: rank), solution} | 2^(n - r)
  solutions and undetermined vars = d
template<int N> pair<int, vector<int>> xor_gaussian_elimination(int
  n, vector<bitset<N + 1>> a, int d) {
  int m = a.size(), r = 0;
  vector<int> where(n, -1);
  for (int c = 0; c < n and r < m; ++c) {
    int p = r;
    while (p < m and not a[p][c]) {
      p++;
    }
    if (p == m) continue;
    swap(a[r], a[p]);
    for (int i = 0; i < m; ++i) {
      if (i != r and a[i][c]) a[i] ^= a[r];
    }
    where[c] = r++;
  }
  for (int i = r; i < m; ++i) {
    if (a[i][n]) return {-1, {}};
  }
  vector<int> x(n, d);
  bitset<N + 1> ones;
  for (int j = 0; j < n; ++j) {
    if (where[j] == -1) ones[j] = d;
  }
  for (int j = 0; j < n; ++j) {
    if (where[j] != -1) x[j] = a[where[j]][n] ^ ((a[where[j]] &
      ones).count() & 1);
  }
  return {r, x};
}
```

josephus

```
// Josephus: finds survivor index eliminating every k-th person
// Time: O(N) | Space: O(N)
// Note: returns 0-based index
int josephus(int n, int k) {
  if (n == 1) return 0;
  return (josephus(n - 1, k) + k) % n;
}
```

matrix

```
// Matrix: product and fast power mod | counts walks of length k and
  linear recurrences
// Time: product O(N^3), power O(N^3 log K) | Space: O(N^2)
// Min-Plus: shortest walk with k edges | fill with inf and identity
  diagonal 0, r[i][j] = min(r[i][j], a[i][k] + b[k][j])
// Max-Plus: longest walk with k edges | fill with -inf and identity
  diagonal 0, r[i][j] = max(r[i][j], a[i][k] + b[k][j])
using matrix = vector<vector<int>>;

matrix operator*(const matrix &a, const matrix &b) {
  int n = a.size();
  int m = b[0].size();
  int h = b.size();
  matrix r(n, vector<int>(m));
  for (int i = 0; i < n; ++i) {
    for (int k = 0; k < h; ++k) {
      for (int j = 0; j < m; ++j) {
        r[i][j] = (r[i][j] + a[i][k] * b[k][j]) % mod;
      }
    }
  }
  return r;
}

matrix mod_pow(matrix a, int e) {
  int n = a.size();
  matrix r(n, vector<int>(n));
  for (int i = 0; i < n; ++i) {
    r[i][i] = 1;
  }
  while (e != 0) {
    if (e & 1) r = r * a;
    a = a * a;
    e >>= 1;
  }
  return r;
}
```

nim

```
bool nim(vector<int> &piles, vector<int> &moves) {
  int n = *max(piles.begin(), piles.end());
  vector<int> grundy(n + 1);

  for (int i = 1; i <= n; ++i) {
    set<int> reach;
    for (int move : moves) if (move <= i) {
      reach.insert(grundy[i - move]);
    }

    int mex = 0;
    while (reach.contains(mex)) mex++;
    grundy[i] = mex;
  }

  int answer = 0;
  for (int pile : piles) {
    answer ^= grundy[pile];
  }
  return answer != 0;
}

bool misere(vector<int> &piles, vector<int> &moves) {
  int n = *max(piles.begin(), piles.end());
  vector<int> grundy(n + 1);

  for (int i = 1; i <= n; ++i) {
    set<int> reach;
    for (int move : moves) if (move <= i) {
      reach.insert(grundy[i - move]);
    }

    int mex = 0;
    while (reach.contains(mex)) mex++;
    grundy[i] = mex;
  }

  bool all_one = true;
  int answer = 0;
  for (int pile : piles) {
    all_one &= grundy[pile] == 1;
    answer ^= grundy[pile];
  }
  return all_one ? (piles.size() & 1) == 0 : answer != 0;
}
```

xor-basis

```
// XOR Basis: linear basis for subset XOR representation and query
  operations
// Time: insert/contains/max/kth O(log) | Space: O(log) where log is
  the maximum bit length (30)
// Note: kth(k) returns k-th smallest distinct XOR | operator[k]
  returns k-th smallest among all 2^n subsets (with duplicates)
struct xor_basis {
  static constexpr int log = 30;
  int n = 0, r = 0;
  array<int, log> basis = {};

  void insert(int x) {
    for (int i = log - 1; i >= 0; --i) if ((x >> i) & 1) {
      if (basis[i] == 0) {
        for (int j = i - 1; j >= 0; --j) if ((x >> j) & 1) {
          x ^= basis[j];
        }
        basis[i] = x;
        r += 1;
        for (int j = log - 1; j > i; --j) if ((basis[j] >> i) & 1) {
          basis[j] ^= x;
        }
        break;
      } else {
        x ^= basis[i];
      }
    }
    n += 1;
  }

  bool contains(int x) {
    for (int i = log - 1; i >= 0; --i) if ((x >> i) & 1) {
      x ^= basis[i];
    }
    return x == 0;
  }

  int max() {
```

```
    int x = 0;
    for (int i = log - 1; i >= 0; --i) if (((x >> i) & 1) == 0) {
      x ^= basis[i];
    }
    return x;
  }

  int kth(int k) {
    if ((k >> r) != 0) return -1;
    int x = 0;
    for (int i = 0; i < log; ++i) if (basis[i] != 0) {
      if (k & 1) x ^= basis[i];
      k >>= 1;
    }
    return x;
  }

  int operator[](int k) {
    if (n - r > 60) return 0;
    return kth(k >> (n - r));
  }

  int ways(int x) {
    if (not contains(x)) return 0;
    return mod_pow(2, n - r);
  }
};
```

number-theory

floor-ceil

```
int floor(int x, int y) {
  return (x / y) - (((x ^ y) < 0) & x % y != 0);
}

int ceil(int x, int y) {
  return (x / y) + (((x ^ y) >= 0) & x % y != 0);
}
```

floor-sum

```
// Floor Sum: calculates sum_{i=0}^{n-1} floor((a * i + b) / c)
// Time: O(log c) | Space: O(log c)
// Note: requires n >= 0 and c >= 1
int floor_sum(int n, int a, int b, int c) {
  if (n == 0) return 0;
  int x = (a % c + c) % c, y = (b % c + c) % c;
  int r = n * (n - 1) / 2 * ((a - x) / c) + n * ((b - y) / c);
  int t = x * n + y;
  return r + floor_sum(t / c, c, t % c, x);
}

// Floor Sum Range: calculates sum_{i=l}^{r-1} floor((a * i + b) / c)
// Time: O(log c) | Space: O(log c)
// Note: requires l < r and c >= 1
int floor_sum_range(int l, int r, int a, int b, int c) {
  return floor_sum(r - l, a, a * l + b, c);
}
```

harmonic-lemma

```
// Harmonic Lemma: quotient blocks in a specific range [l, r]
// Time: O(sqrt(N)) | Space: O(1)
// Note: iterates over ranges [i, j] inside [l, r] where floor(n / k)
  is exactly q
int harmonic_lemma(int n, int l, int r) {
  int s = 0;
  for (int i = l, j; i <= r; i = j + 1) {
    int q = n / i;
    if (q == 0) {
      j = r;
    } else {
      j = min(r, n / q);
    }
    s += q * (j - i + 1);
  }
  return s;
}
```

mobius

```
int mobius(int n) {
  int mu = 1;
  for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
    n /= i;
    if (n % i == 0) return 0;
    mu *= -1;
  }
  if (n > 1) mu *= -1;
  return mu;
}

vector<int> mobius_sieve(int n) {
  vector<int> mu(n + 1);
  vector<bool> is_prime(n + 1, true);
  vector<int> primes;
  mu[1] = 1;
  is_prime[1] = false;
  for (int i = 2; i <= n; ++i) {
    if (is_prime[i]) {
      primes.push_back(i);
      mu[i] = -1;
    }
    for (int p : primes) {
      if (i * p > n) break;
      is_prime[i * p] = false;
      if (i % p == 0) {
        mu[i * p] = 0;
        break;
      } else {
        mu[i * p] = -mu[i];
      }
    }
  }
  return mu;
}
```

chinese-remainder-theorem

mod/

```
int128 chinese_remainder_theorem(const vector<int> &rem, const
  vector<int> &mod) {
  int128 r = rem[0], m = mod[0];
  for (int i = 1; i < rem.size(); ++i) {
    int g = gcd(static_cast<int>(m % mod[i]), mod[i]), h = mod[i] / g;
    if ((rem[i] - r) % g != 0) return -1;

    auto [x, _] = extended_gcd(static_cast<int>(m / g % h), h);

    r += m * (((((rem[i] - r) / g) * x) % h + h) % h);
    m *= h;
  }
  return r;
}
```

extended-gcd

mod/

```
// Extended GCD: finds coefficients x and y such that ax + by =
  gcd(a, b)
// Time: O(log(min(a, b))) | Space: O(log(min(a, b)))
// Note: returns [x0, y0] | all valid pairs are: x = x0 + k*(b/gcd)
  and y = y0 - k*(a/gcd)
pair<int, int> extended_gcd(int a, int b) {
  if (b == 0) return {1, 0};

  auto [x, y] = extended_gcd(b, a % b);
  return {y, x - (a / b) * y};
}
```

mod-log

mod/

```
int mod_log(int a, int b, int mod) {
  if (a == 0) {
    if (b == 0) return 1 % mod;
    return -1;
  }

  int k = 1, h = 0;
  for (int g = gcd(a, mod); g != 1; g = gcd(a, mod), ++h) {
    if (k == b) return h;
    if (b % g) return -1;

    b /= g, mod /= g;
    k = (k * (a / g)) % mod;
  }

  int n = ceil(sqrt(mod)), s = 1;
  for (int i = 1; i <= n; ++i) {
    s = (s * a) % mod;
  }

  map<int, int> f;
  for (int q = 0; q <= n; ++q) {
    f[b] = q;
    b = (b * a) % mod;
  }

  a = (k * s) % mod;
  for (int p = 1; p <= n; ++p) {
    if (f.contains(a)) {
      return (n * p - f[a]) + h;
    }
    a = (a * s) % mod;
  }
  return -1;
}
```

mod-pow

mod/

```
int mod_pow(int a, int e, const int mod) {
  int r = 1;
  while (e != 0) {
    if (e & 1) r = (r * a) % mod;
    a = (a * a) % mod;
    e >>= 1;
  }
  return r;
}
```

multi-extended-gcd

mod/

```
vector<int> multi_extended_gcd(vector<int> &a) {
  int n = a.size(), g = a[0];
  vector<int> c(n, 1), h(n - 1);

  for (int i = 1; i < n; ++i) {
    auto [x, y] = extended_gcd(g, a[i]);
    g = gcd(g, a[i]);
    h[i - 1] = x;
    c[i] = y;
  }

  int p = 1;
  for (int i = n - 1; i > 0; --i) {
    c[i] *= p;
    p *= h[i - 1];
  }
  c[0] *= p;
  return c;
}
```

linear-sieve

prime/

```
vector<int> linear_sieve(int n) {
  vector<int> p, spf(n + 1);
  for (int i = 2; i <= n; ++i) {
    if (spf[i] == 0) {
      spf[i] = i;
      p.push_back(i);
    }
    for (int j : p) {
      if (j > spf[i] or i * j > n) break;
      spf[i * j] = j;
    }
  }
  return p;
}
```

miller-rabin

prime/

```
int mod_pow(int128 a, int e, const int mod) {
  int128 r = 1;
  while (e != 0) {
    if (e & 1) r = (r * a) % mod;
    a = (a * a) % mod;
    e >>= 1;
  }
  return r;
}

bool composite(int n, int a, int s, int d) {
  int128 x = mod_pow(a, d, n);
  if (x == 1 or x == n - 1) return false;
  for (int i = 1; i < s; ++i) {
    x = (x * x) % n;
    if (x == n - 1) return false;
    if (x == 1) return true;
  }
  return true;
}

bool miller_rabin(int n) {
  if (n <= 1) return false;
  static const int P[] = {2, 3, 5, 13, 19, 73, 193, 407521,
    299210837};
  for (int p : P) {
    if (n == p) return true;
    if (n % p == 0) return false;
  }
  int s = __builtin_ctzll(n - 1);
  int d = (n - 1) >> s;
  static const int A[] = {2, 325, 9375, 28178, 450775, 9780504,
    1795265022};
  for (int a : A) {
    if (composite(n, a, s, d)) return false;
  }
  return true;
}
```

pollard-rho

prime/

```
// Pollard-Rho: fast integer factorization
// Time: O(N^(1/4)) expected | Space: O(1)
// Note: finds a non-trivial divisor | use recursively to fully factor
mt19937_64
  rng(chrono::steady_clock::now().time_since_epoch().count());

int f(int128 x, int c, const int mod) {
```

```
  return (x * x + c) % mod;
}

int pollard_rho(int n) {
  if ((n & 1) == 0) return 2;
  if (miller_rabin(n)) return n;
  int c = uniform_int_distribution<int>(1, n - 1)(rng);
  int x = 2, y = 2, g = 1, l = 1, p = 2, q = 1;
  while (g == 1) {
    y = x;
    int k = 0;
    while (k < l and g == 1) {
      p = x;
      for (int i = 1; i <= min<int>(128, l - k); ++i) {
        x = f(x, c, n);
        q = ((int128) q * abs(x - y)) % n;
      }
      g = gcd(q, n);
      k += 128;
    }
    l *= 2;
  }
  if (g == n) {
    do {
      p = f(p, c, n);
      g = gcd(abs(p - y), n);
    } while (g == 1);
  }
  if (g == n) return pollard_rho(n);
  return g;
}
```

segmented-sieve

prime/

```
vector<int> segmented_sieve(int l, int r) {
  vector<int> primes = linear_sieve(sqrt(r));
  offset_array<char> is_prime(l, r + 1, true);
  for (int p : primes) {
    for (int i = max(p * ((l + p - 1) / p), p * p); i <= r; i += p) {
      is_prime[i] = false;
    }
  }
  primes.clear();
  for (int i = max(2, l); i <= r; ++i) {
    if (is_prime[i]) {
      primes.push_back(i);
    }
  }
  return primes;
}
```

totient

```
int totient(int n) {
  int phi = n;
  for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
    phi -= phi / i;
    while (n % i == 0) n /= i;
  }
  if (n > 1) phi -= phi / n;
  return phi;
}

vector<int> totient_sieve(int n) {
  vector<int> phi(n + 1);
  for (int i = 1; i <= n; ++i) {
    phi[i] = i;
  }
  for (int i = 2; i <= n; ++i) if (phi[i] == i) {
    for (int j = i; j <= n; j += i) {
      phi[j] -= phi[j] / i;
    }
  }
  return phi;
}
```

square-root

decomposition

```
struct decomposition {
  int n, m;
  vector<int> a, b;

  decomposition(vector<int> &base, float f = 1.0) {
    n = base.size();
    m = clamp<int>(f * sqrt(n), 1, n);
    a.resize(n);
    b.resize((n + (m - 1)) / m);
    for (int i = 0; i < n; ++i) {
      a[i] = base[i];
      b[i / m] += a[i];
    }
  }

  void update(int i, int x) {
    a[i] = x;
    int b_i = i / m;
    b[b_i] = 0;
    int l = b_i * m;
    int r = min(n, (b_i + 1) * m) - 1;
    for (int k = l; k <= r; ++k) {
      b[b_i] += a[k];
    }
  }

  int query(int l, int r) {
    int s = 0;
    int b_l = l / m;
    int b_r = r / m;
    if (b_l == b_r) {
      for (int i = l; i <= r; ++i) {
        s += a[i];
      }
    } else {
      for (int i = l; i < (b_l + 1) * m; ++i) {
        s += a[i];
      }
      for (int i = b_l + 1; i < b_r; ++i) {
        s += b[i];
      }
      for (int i = b_r * m; i <= r; ++i) {
        s += a[i];
      }
    }
    return s;
  }
};
```

mo-3d

```
// Mo's Algorithm 3D: offline range queries with point updates
// Time: O(N^(5/3)) | Space: O(N + Q + U)
// Q elements: {l, r, t, id} | t = # of updates prior to query | id =
  original query index
// U elements: {i, x} | i = array index to update | x = new value
// M parameter: block size multiplier to tune the N^(2/3) block size
// Note: implement 'insert', 'remove', and 'curr' state according to
  your specific problem
vector<int> mo_3d(vector<int> &a, vector<array<int, 4>> &q,
  vector<array<int, 2>> &u, float m = 1.0) {
  int b = clamp<int>(m * pow(a.size(), 2.0 / 3.0), 1, a.size());
  auto block = [&](int i) -> int {
    return i / b;
  };

  sort(q.begin(), q.end(), [&](const array<int, 4> &x, const
    array<int, 4> &y) {
    if (block(x[0]) != block(y[0])) return block(x[0]) < block(y[0]);
    if (block(x[1]) != block(y[1])) return (block(x[0]) & 1) ?
      block(x[1]) < block(y[1]) : block(x[1]) > block(y[1]);
    return (block(x[1]) & 1) ? x[2] < y[2] : x[2] > y[2];
  });

  int curr = 0;
  auto insert = [&](int x) -> void {};
  auto remove = [&](int x) -> void {};

  auto update = [&](int t, int l, int r) -> void {
    int i = u[t][0];
    if (l <= i and i <= r) {
      remove(a[i]);
      swap(a[i], u[t][1]);
      insert(a[i]);
    } else {
      swap(a[i], u[t][1]);
    }
  };

  vector<int> ans(q.size());
  int h = 0, i = 0, j = -1;
  for (auto &[l, r, t, id] : q) {
    while (h < t) update(h++, i, j);
    while (h > t) update(--h, i, j);
    while (i > l) insert(a[--i]);
    while (j < r) insert(a[++j]);
    while (i < l) remove(a[i++]);
    while (j > r) remove(a[j--]);
    ans[id] = curr;
  }
  return ans;
}
```

mo

```
int hilbert(int x, int y) {
  int h = 0;
  for (int s = 25; s >= 0; --s) {
    int rx = (x >> s) & 1;
    int ry = (y >> s) & 1;
    int qd = (3 * rx) ^ ry;
    h = (h << 2) | qd;
    if (ry == 0) {
      x ^= -rx;
      y ^= -rx;
      swap(x, y);
    }
  }
  return h;
}

vector<int> mo(vector<int> &a, vector<array<int, 4>> &q) {
  for (auto &[h, l, r, id] : q) h = hilbert(l, r);
  sort(q.begin(), q.end());

  int curr = 0;
  auto insert = [&](int x) -> void {};
  auto remove = [&](int x) -> void {};

  vector<int> ans(q.size());
  int i = 0, j = -1;
  for (auto &[h, l, r, id] : q) {
    while (i > l) insert(a[--i]);
    while (j < r) insert(a[++j]);
    while (i < l) remove(a[i++]);
    while (j > r) remove(a[j--]);
    ans[id] = curr;
  }
  return ans;
}
```

string

aho-corasick

```
struct aho_corasick {
  struct node {
    int fail = 0;
    int exit = 0;
    int len = 0;
    int cnt = 0;
    array<int, 26> next = {};
  };

  vector<node> trie;

  aho_corasick(int n = 0) {
    trie.reserve(n + 5);
    trie.emplace_back();
  }

  void insert(string &s) {
    int i = 0;
    for (char c : s) {
      int k = c - 'a';
      if (trie[i].next[k] == 0) {
        trie[i].next[k] = trie.size();
        trie.emplace_back();
      }
      i = trie[i].next[k];
    }
    trie[i].len = s.length();
    trie[i].cnt++;
  }

  void compile() {
    queue<int> q;
    for (int c = 0; c < 26; ++c) {
      int i = trie[0].next[c];
      if (i != 0) q.push(i);
    }
    while (q.size()) {
      int i = q.front(); q.pop();
      int f = trie[i].fail;
      trie[i].exit = trie[f].cnt != 0 ? f : trie[f].exit;
      for (int c = 0; c < 26; ++c) {
        int &j = trie[i].next[c];
        if (j != 0) {
          trie[j].fail = trie[f].next[c];
          q.push(j);
        } else {
          j = trie[f].next[c];
        }
      }
    }
  }

  vector<int> match(string &t) {
    int n = t.length();
    vector<int> p;
    int i = 0;
```

```
    for (int k = 0; k < n; ++k) {
      i = trie[i].next[t[k] - 'a'];
      for (int j = i; j != 0; j = trie[j].exit) if (trie[j].cnt != 0)
        {
        for (int r = 1; r <= trie[j].cnt; ++r) {
          p.push_back(k - trie[j].len + 1);
        }
      }
    }
    return p;
  }
};
```

deterministic-finite-automaton

```
vector<array<int, 26>> deterministic_finite_automaton(const string
  &s) {
  int n = s.length();
  vector<array<int, 26>> dfa(n + 1);
  if (n == 0) return dfa;
  dfa[0][s[0] - 'a'] = 1;
  for (int i = 1, j = 0; i <= n; ++i) {
    for (int c = 0; c < 26; ++c) {
      dfa[i][c] = dfa[j][c];
    }
    if (i < n) {
      dfa[i][s[i] - 'a'] = i + 1;
      j = dfa[j][s[i] - 'a'];
    }
  }
  return dfa;
}
```

dynamic-hasher

```
block init(char c) {
  block h;
  h.len = 1;
  for (int i = 0; i <= 1; ++i) {
    h.ord[i] = c % mod[i];
    h.rev[i] = c % mod[i];
  }
  return h;
}

block combine(const block &l, const block &r) {
  if (l.len == 0) return r;
  if (r.len == 0) return l;
  extend(max(l.len, r.len));
  block m;
  m.len = l.len + r.len;
  for (int i = 0; i <= 1; ++i) {
    m.ord[i] = (l.ord[i] * pows[r.len][i] + r.ord[i]) % mod[i];
    m.rev[i] = (r.rev[i] * pows[l.len][i] + l.rev[i]) % mod[i];
  }
  return m;
}

struct dynamic_hasher {
  int n;
  vector<block> tree;

  dynamic_hasher(const string &s): n(s.length()), tree(2 * n) {
    for (int i = 0; i < n; ++i) tree[n + i] = init(s[i]);
    for (int i = n - 1; i > 0; --i) tree[i] = combine(tree[i << 1],
      tree[(i << 1) | 1]);
  }

  void update(int i, char c) {
    tree[i += n] = init(c);
    for (i >>= 1; i > 0; i >>= 1) tree[i] = combine(tree[i << 1],
      tree[(i << 1) | 1]);
  }

  block query(int i, int j) {
    block l, r;
    for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
      if (i & 1) l = combine(l, tree[i++]);
      if (j & 1) r = combine(tree[--j], r);
    }
    return combine(l, r);
  }
};
```

hasher

```
const array<int, 2> base = {41, 53};
const array<int, 2> mod = {1000000007, 1000000009};

vector<array<int, 2>> pows = {{{1, 1}}};

void extend(int n) {
  while (pows.size() <= n) {
    array<int, 2> back = pows.back();
    array<int, 2> curr = {(back[0] * base[0]) % mod[0], (back[1] *
      base[1]) % mod[1]};
    pows.push_back(curr);
  }
}

struct block {
  int len = 0;
  array<int, 2> ord = {};
  array<int, 2> rev = {};

  bool operator==(const block &o) const {
    return len == o.len and ord == o.ord;
  }
};

struct hasher {
  int n;
  vector<array<int, 2>> pref, suf;

  hasher(const string &s): n(s.length()), pref(n + 1), suf(n + 2) {
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j <= 1; ++j) {
        pref[i + 1][j] = (pref[i][j] * base[j] + s[i]) % mod[j];
        suf[n - i][j] = (suf[n - i + 1][j] * base[j] + s[n - i - 1])
          % mod[j];
      }
    }
    extend(n);
  }

  block query(int l, int r) const {
    block h;
    h.len = r - l + 1;
    for (int i = 0; i <= 1; ++i) {
      h.ord[i] = (pref[r + 1][i] - pref[l][i] * pows[h.len][i]) %
        mod[i];
      h.ord[i] = (h.ord[i] + mod[i]) % mod[i];
      h.rev[i] = (suf[l + 1][i] - suf[r + 2][i] * pows[h.len][i]) %
        mod[i];
      h.rev[i] = (h.rev[i] + mod[i]) % mod[i];
    }
    return h;
  }
};
```

longest-prefix-suffix

```
vector<int> longest_prefix_suffix(string &s) {
  int n = s.length();
  vector<int> lps(n);
  for (int i = 1; i < n; ++i) {
    int j = lps[i - 1];
    while (j != 0 and s[i] != s[j]) j = lps[j - 1];
    if (s[i] == s[j]) j++;
    lps[i] = j;
  }
  return lps;
}
```

manacher

```
string pre_process(string &s) {
  string t = "#";
  for (char &c : s) {
    t.push_back(c);
    t.push_back('#');
  }
  return t;
}

vector<int> manacher(string &s) {
  string t = '^' + pre_process(s) + '$';
  int n = t.length();

  vector<int> p(n);
  int c = 0, r = 0;
  for (int i = 1; i < n - 1; i++) {
    int j = 2 * c - i;
    if (i < r) {
      p[i] = min(r - i, p[j]);
    }
    while (t[i + (p[i] + 1)] == t[i - (p[i] + 1)]) {
      p[i]++;
    }
    if (i + p[i] > r) {
      c = i;
      r = i + p[i];
    }
  }
  return p;
}

string longest_palindrome(string &s) {
  vector<int> p = manacher(s);
  int n = p.size();

  int k = 0, l = 0;
  for (int i = 1; i < n - 1; i++) {
    if (p[i] > l) {
      l = p[i];
      k = (i - l) / 2;
    }
  }
  return s.substr(k, l);
}

int count_palindromes(string &s) {
  vector<int> p = manacher(s);
  int n = p.size();

  int total = 0;
  for (int i = 1; i < n - 1; ++i) {
    total += (p[i] + 1) / 2;
  }
  return total;
}
```

palindromic-tree

```
struct palindromic_tree {
  struct node {
    int length, fail;
    array<int, 26> child = {};

    node(int length): length(length), fail(0) {}
  };

  int suffix;
  vector<char> text;
  vector<node> tree;

  palindromic_tree(int n): suffix(0) {
    text.reserve(n + 5);
    text.push_back('$');
    tree.reserve(n + 5);
    tree.push_back(node(0));
    tree.push_back(node(-1));
    tree[0].fail = 1;
  }

  int follow(int id) {
    while (text[(text.size() - tree[id].length) - 2] != text.back()) {
      id = tree[id].fail;
    }
    return id;
  }

  void insert(char c) {
    text.push_back(c);
    suffix = follow(suffix);

    if (not tree[suffix].child[c - 'a']) {
      int id = tree.size();
      tree.push_back(node(tree[suffix].length + 2));

      tree[id].fail = tree[follow(tree[suffix].fail)].child[c - 'a'];
      tree[suffix].child[c - 'a'] = id;
    }
    suffix = tree[suffix].child[c - 'a'];
  }
};
```

suffix-array

```
// Suffix Array: lexicographical sorting of all string suffixes
// Time: build O(N log N) | LCP O(N) | search O(|P| log N) | Space:
  O(N)
// Note: expects sentinel character at the end of input | supports
  pattern matching and k-th queries
struct suffix_array {
  int n, m;
  string s;
  vector<int> suf, lcp;
  vector<int> rank, temp;

  void sort() {
    vector<int> f(m + 1);
    for (int i = 0; i < n; ++i) {
      int r = rank[suf[i]];
      f[r] += 1;
    }
    for (int i = 1; i <= m; ++i) {
      f[i] += f[i - 1];
    }
    for (int i = n - 1; i >= 0; --i) {
      int r = rank[suf[i]];
```

```
    f[r] -= 1;
    temp[f[r]] = suf[i];
  }
  swap(suf, temp);
}

suffix_array(string &s): n(s.length()), m(0), s(s), suf(n), lcp(n),
 rank(n), temp(n) {
  for (int i = 0; i < n; ++i) {
    suf[i] = i;
    rank[i] = s[i];
    m = max<int>(m, s[i]);
  }
  sort();
  for (int k = 1; k < n; k <<= 1) {
    for (int i = 0; i < n; ++i) {
      suf[i] = (suf[i] - k + n) % n;
    }
    sort();
    temp[suf[0]] = 1;
    for (int i = 1; i < n; ++i) {
      int prev = suf[i - 1];
      int curr = suf[i];
      if (rank[prev] == rank[curr] and rank[(prev + k) % n] ==
       rank[(curr + k) % n]) {
        temp[curr] = temp[prev];
      } else {
        temp[curr] = temp[prev] + 1;
      }
    }
    swap(rank, temp);
    m = rank[suf[n - 1]];
    if (m == n) break;
  }
}

void longest_common_prefix() {
  vector<int> inv(n);
  for (int i = 0; i < n; ++i) {
    inv[suf[i]] = i;
  }
  int l = 0;
  for (int i = 0; i < n; ++i) {
    if (inv[i] == 0) {
      l = 0;
      continue;
    }
    int j = suf[inv[i] - 1];
    while (i + l < n and j + l < n and s[i + l] == s[j + l]) l++;
    lcp[inv[i]] = l;
    l = max<int>(0, l - 1);
  }
}

int compare(int i, string &t) {
  return s.compare(i, t.size(), t);
}

int lower_search(string &t) {
  int answer = 0;
  int low = 1, high = n - 1;
  while (low <= high) {
    int mid = (low + high) / 2;
    if (compare(suf[mid], t) < 0) {
      low = mid + 1;
    } else {
      answer = mid;
      high = mid - 1;
    }
  }
  return answer;
}

int upper_search(string &t) {
  int answer = n;
  int low = 1, high = n - 1;
  while (low <= high) {
    int mid = (low + high) / 2;
    if (compare(suf[mid], t) <= 0) {
      low = mid + 1;
    } else {
      answer = mid;
      high = mid - 1;
    }
  }
  return answer;
}

int search(string &t) {
  int lower = lower_search(t);
  if (compare(suf[lower], t) != 0) return 0;
  int upper = upper_search(t);
  return upper - lower;
}

pair<int, int> kth_distinct_substring(int k) {
  assert(k >= 0);
  for (int i = 1; i < n; ++i) {
    int len = (n - 1) - suf[i];
    int sub = len - lcp[i];
    if (k < sub) return {suf[i], lcp[i] + k + 1};
    k -= sub;
  }
  return {-1, -1};
}

pair<int, int> kth_substring(int k) {
  assert(k >= 0);
  sparse_table st(lcp);
  for (int i = 1; i < n; ++i) {
    int curr_len = lcp[i] + 1;
    int max_len = (n - 1) - suf[i];
    while (curr_len <= max_len) {
      int j = i;
      int low = i + 1, high = n - 1;
      while (low <= high) {
        int mid = (low + high) / 2;
        if (st.query(i + 1, mid) >= curr_len) {
          j = mid;
          low = mid + 1;
        } else {
          high = mid - 1;
        }
      }
      int freq = j - i + 1;
      int len = max_len;
      if (j > i) len = min(len, st.query(i + 1, j));
      int sub = (len - curr_len + 1) * freq;
      if (k < sub) return {suf[i], curr_len + (k / freq)};
      k -= sub;
      curr_len = len + 1;
    }
  }
  return {-1, -1};
}
};
```

suffix-automaton

```
// Suffix Automaton: directed acyclic graph representing all
 substrings of a string
// Time: extend O(1) amortized | Space: O(N * S)
// Note: root at index 1 | alphabet [a-z]
struct suffix_automaton {
  struct node {
    int len = 0, link = 0;
    int pos = -1, cnt = 0;
    array<int, 26> next = {};
  };

  int prev;
  vector<node> st;

  suffix_automaton(): prev(1), st(2) {}

  void extend(char c) {
    int idx = c - 'a';
    int curr = st.size();
    int ptr = prev;

    st.emplace_back();
    st[curr].len = st[prev].len + 1;
    st[curr].pos = st[curr].len - 1;
    st[curr].cnt = 1;
    prev = curr;

    while (ptr and (not st[ptr].next[idx])) {
      st[ptr].next[idx] = curr;
      ptr = st[ptr].link;
    }

    if (not ptr) {
      st[curr].link = 1;
      return;
    }

    int qtr = st[ptr].next[idx];
    if (st[ptr].len + 1 == st[qtr].len) {
      st[curr].link = qtr;
      return;
    }

    int clone = st.size();
    st.push_back(st[qtr]);
    st[clone].len = st[ptr].len + 1;
    st[clone].cnt = 0;

    while (ptr and st[ptr].next[idx] == qtr) {
      st[ptr].next[idx] = clone;
      ptr = st[ptr].link;
    }

    st[curr].link = st[qtr].link = clone;
  }

  bool contains(string &t) {
    int ptr = 1;
    for (char c : t) {
      int idx = c - 'a';
      ptr = st[ptr].next[idx];
      if (not ptr) return false;
    }
    return true;
  }
};
```

tree-suffix-array

```
// Tree Suffix Array: lexicographical sorting of all tree paths (node
 to root)
// Time: build O(N log N) | LCP O(log N) | search O(|P| log N) |
 Space: O(N log N)
// Note: 1-indexed nodes, node 0 is reserved as a sentinel root with
 s[0] = '$'
struct tree_suffix_array {
  int n, m, log;
  string s;
  vector<int> suf, lcp, temp, depth, freq;
  vector<vector<int>> up, rank;

  void dfs(int a, int p, int d, vector<vector<int>> &g) {
    up[0][a] = p;
    depth[a] = d;
    for (int k = 1; k <= log; ++k) {
      up[k][a] = up[k - 1][up[k - 1][a]];
    }
    for (int b : g[a]) if (b != p) {
      dfs(b, a, d + 1, g);
    }
  }

  void sort(int k, bool jump) {
    fill(freq.begin(), freq.begin() + m + 1, 0);
    for (int i = 0; i < n; ++i) {
      int h = suf[i];
      if (jump) h = up[k][h];
      int r = rank[k][h];
      freq[r]++;
    }
    for (int i = 1; i <= m; ++i) {
      freq[i] += freq[i - 1];
    }
    for (int i = n - 1; i >= 0; --i) {
      int h = suf[i];
      if (jump) h = up[k][h];
      int r = rank[k][h];
      freq[r]--;
      temp[freq[r]] = suf[i];
    }
    swap(suf, temp);
  }

  tree_suffix_array(const string &letters, vector<vector<int>> &g):
   n(letters.length()), m(0), log(__lg(n | 1) + 1),
  s(letters), suf(n), lcp(n), temp(n), depth(n), freq(max<int>(256,
   n)), up(log + 1, vector<int>(n)), rank(log + 1, vector<int>(n)) {
    dfs(1, 0, 1, g);
    for (int i = 0; i < n; ++i) {
      suf[i] = i;
      rank[0][i] = s[i];
      m = max<int>(m, s[i]);
    }
    for (int k = 0; k < log; ++k) {
      sort(k, true);
      sort(k, false);
      rank[k + 1][suf[0]] = 0;
      int val = 0;
      for (int i = 1; i < n; ++i) {
        int prev = suf[i - 1];
        int curr = suf[i];
        int j_prev = up[k][prev];
        int j_curr = up[k][curr];
        if (rank[k][prev] == rank[k][curr] and rank[k][j_prev] ==
         rank[k][j_curr]) {
          rank[k + 1][curr] = val;
```

```
        } else {
          rank[k + 1][curr] = ++val;
        }
      }
      m = val;
      if (m == n - 1) {
        for (int h = k + 1; h <= log; ++h) rank[h] = rank[k + 1];
        break;
      }
    }
  }

  int longest_common_prefix(int a, int b) {
    if (a == b) return depth[a];
    int answer = 0;
    for (int k = log; k >= 0 and a != 0 and b != 0; --k) if
      (rank[k][a] == rank[k][b]) {
      answer += 1 << k;
      a = up[k][a];
      b = up[k][b];
    }
    return answer;
  }

  void build_longest_common_prefix() {
    for (int i = 1; i < n; ++i) {
      lcp[i] = longest_common_prefix(suf[i], suf[i - 1]);
    }
  }

  int compare(int a, string &t) {
    for (int i = 0; i < t.length(); ++i) {
      if (a == 0) return -1;
      if (s[a] != t[i]) return s[a] < t[i] ? -1 : +1;
      a = up[0][a];
    }
    return 0;
  }

  int lower_search(string &t) {
    int answer = 0;
    int low = 1, high = n - 1;
    while (low <= high) {
      int mid = (low + high) / 2;
      if (compare(suf[mid], t) < 0) {
        low = mid + 1;
      } else {
        answer = mid;
        high = mid - 1;
      }
    }
    return answer;
  }

  int upper_search(string &t) {
    int answer = n;
    int low = 1, high = n - 1;
    while (low <= high) {
      int mid = (low + high) / 2;
      if (compare(suf[mid], t) <= 0) {
        low = mid + 1;
      } else {
        answer = mid;
        high = mid - 1;
      }
    }
    return answer;
  }

  int search(string &t) {
    int lower = lower_search(t);
    if (compare(suf[lower], t) != 0) return 0;
    int upper = upper_search(t);
    return upper - lower;
  }
};
```

trie

```
struct trie {
  struct node {
    int pref = 0;
    int end = 0;
    array<int, 26> child = {};
  };

  vector<node> tree;

  trie(int n = 0) {
    tree.reserve(n + 1);
    tree.emplace_back();
  }

  void insert(string &s) {
    int i = 0;
    tree[0].pref++;
    for (char c : s) {
      int k = c - 'a';
      if (tree[i].child[k] == 0) {
        tree[i].child[k] = tree.size();
        tree.emplace_back();
      }
      i = tree[i].child[k];
      tree[i].pref++;
    }
    tree[i].end++;
  }

  void remove(string &s) {
    if (not contains(s)) return;
    int i = 0;
    tree[i].pref--;
    for (char c : s) {
      int k = c - 'a';
      i = tree[i].child[k];
      tree[i].pref--;
    }
    tree[i].end--;
  }

  bool contains(string &s) {
    int i = 0;
    for (char c : s) {
      int k = c - 'a';
      i = tree[i].child[k];
      if (i == 0) return false;
    }
    return tree[i].end != 0;
  }
};
```

z-array

```
vector<int> z_array(string &s) {
  int n = s.length();
  vector<int> z(n);
  for (int i = 1, l = 0, r = 0; i < n; ++i) {
    z[i] = max(0, min(z[i - l], r - (i + 1)));
    while (i + z[i] < n and s[i + z[i]] == s[z[i]]) {
      l = i;
      r = i + z[i];
      z[i] += 1;
    }
  }
  return z;
}
```

structs

binary-indexed-tree

binary-indexed-tree/

```
struct bit {
    int n;
    vector<int> tree;

    static int lsb(int i) {
        return i & -i;
    }

    bit(int n = 0): n(n), tree(n + 1) {}

    void update(int i, int x) {
        for (i += 1; i <= n; i += lsb(i)) tree[i] += x;
    }

    int prefix(int i) {
        int s = 0;
        for (i += 1; i > 0; i -= lsb(i)) s += tree[i];
        return s;
    }

    int query(int i, int j) {
        return prefix(j) - prefix(i - 1);
    }

    int find_kth(int k) {
        assert(k >= 0);
        k += 1;
        int i = 0;
        for (int s = 1 << __lg(n); s > 0; s >>= 1) if (i + s <= n and
          tree[i + s] < k) {
            i += s;
            k -= tree[i];
        }
        return i;
    }
};

struct bit_2d {
    int n, m;
    vector<bit> tree;

    bit_2d(int n = 0, int m = 0): n(n), m(m), tree(n + 1, bit(m)) {}

    void build(vector<vector<int>> &a) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                update(i, j, a[i][j]);
            }
        }
    }

    void update(int i, int j, int x) {
        for (i += 1; i <= n; i += i & -i) tree[i].update(j, x);
    }

    int prefix(int i, int j) {
        int s = 0;
        for (i += 1; i > 0; i -= i & -i) s += tree[i].prefix(j);
        return s;
    }

    int query(int i, int j, int p, int q) {
        return prefix(p, q)
             - prefix(i - 1, q)
             - prefix(p, j - 1)
             + prefix(i - 1, j - 1);
    }
};
```

range-binary-indexed-tree

binary-indexed-tree/

```
struct range_bit {
  bit m, s;

  range_bit(int n): m(n + 1), s(n + 1) {}

  void update(int i, int j, int x) {
    m.update(i, x);
    m.update(j + 1, -x);
    s.update(i, x * (i - 1));
    s.update(j + 1, -x * j);
  }

  int prefix(int i) {
    return i * m.prefix(i) - s.prefix(i);
  }

  int query(int i, int j) {
    return prefix(j) - prefix(i - 1);
  }
};
```

sparse-binary-indexed-tree

binary-indexed-tree/

```
// Sparse BIT: 2D Binary Indexed Tree with Static Coordinate
  Compression
// Time: Build O(Q log^2 Q) | Update/Query O(log^2 Q) | Space: O(Q
  log Q)
// Queries: {type, i1, j1, i2, j2} | 'U' = Update, 'Q' = Query
struct sparse_bit {
  int n;
  vector<int> is;
  vector<vector<int>> js;
  vector<bit> tree;

  void compress(vector<int> &a) {
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
  }

  int id(vector<int> &a, int i) {
    return lower_bound(a.begin(), a.end(), i) - a.begin();
  }

  sparse_bit(vector<tuple<char, int, int, int, int>> &q) {
    for (auto [t, i1, j1, i2, j2] : q) {
      if (t == 'U') is.push_back(i1);
      else if (t == 'Q') is.push_back(i1 - 1), is.push_back(i2);
    }
    compress(is);
    n = is.size();

    js.resize(n + 1);
    for (auto [t, i1, j1, i2, j2] : q) {
      if (t == 'U') {
        for (i1 = id(is, i1) + 1; i1 <= n; i1 += bit::lsb(i1))
          js[i1].push_back(j1);
      } else if (t == 'Q') {
```

```
        for (i1 = id(is, i1 - 1) + 1; i1 > 0; i1 -= bit::lsb(i1))
          js[i1].push_back(j1 - 1), js[i1].push_back(j2);
        for (i2 = id(is, i2) + 1; i2 > 0; i2 -= bit::lsb(i2))
          js[i2].push_back(j1 - 1), js[i2].push_back(j2);
      }
    }

    tree.resize(n + 1);
    for (int i = 1; i <= n; ++i) {
      compress(js[i]);
      tree[i] = bit(js[i].size());
    }
  }

  void update(int i, int j, int x) {
    for (i = id(is, i) + 1; i <= n; i += bit::lsb(i))
      tree[i].update(id(js[i], j), x);
  }

  int prefix(int i, int j) {
    int s = 0;
    for (i = id(is, i) + 1; i > 0; i -= bit::lsb(i)) s +=
      tree[i].prefix(id(js[i], j));
    return s;
  }

  int query(int i1, int j1, int i2, int j2) {
    return prefix(i2, j2) - prefix(i1 - 1, j2) - prefix(i2, j1 - 1) +
      prefix(i1 - 1, j1 - 1);
  }
};
```

cover-tree

cover-tree/

```
// Cover Tree: calculates total length covered by at least one
  interval
// Time: update O(log N), query O(1) | Space: O(N)
struct cover_tree {
  int n;
  vector<int> all, cnt, curr;

  void recalc(int i) {
    if (cnt[i] > 0) curr[i] = all[i];
    else if (i < n) curr[i] = curr[i << 1] + curr[i << 1 | 1];
    else curr[i] = 0;
  }

  cover_tree(vector<int> &x): n(x.size() - 1), all(2 * n), cnt(2 *
    n), curr(2 * n) {
    for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
    for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1
      | 1];
  }

  void update(int i, int j, int x) {
    int p = i, q = j;
    for (i += n, j += n; i < j; i >>= 1, j >>= 1) {
      if (i & 1) cnt[i] += x, recalc(i++);
      if (j & 1) cnt[--j] += x, recalc(j);
    }
    i = p + n;
    while (i >>= 1) recalc(i);
    j = q + n - 1;
    while (j >>= 1) recalc(j);
  }

  int query() {
    return curr[1];
  }
};

// XOR Cover Tree: calculates total length covered by an odd number
  of intervals
// Time: update O(log N), query O(1) | Space: O(N)
struct xor_cover_tree {
  int n;
  vector<int> all, cnt, curr;

  void recalc(int i) {
    int child = 0;
    if (i < n) child = curr[i << 1] + curr[i << 1 | 1];
    if (cnt[i] & 1) curr[i] = all[i] - child;
    else curr[i] = child;
  }

  xor_cover_tree(vector<int> &x): n(x.size() - 1), all(2 * n), cnt(2
    * n), curr(2 * n) {
    for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
    for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1
      | 1];
  }

  void update(int i, int j) {
    int p = i, q = j;
    for (i += n, j += n; i < j; i >>= 1, j >>= 1) {
      if (i & 1) cnt[i] ^= 1, recalc(i++);
      if (j & 1) cnt[--j] ^= 1, recalc(j);
    }
    i = p + n;
    while (i >>= 1) recalc(i);
    j = q + n - 1;
    while (j >>= 1) recalc(j);
  }

  int query() {
    return curr[1];
  }
};
```

k-cover-tree

cover-tree/

```
// Iterative K-Cover Segment Tree: calculates total length covered by
  at least K intervals
// Time: update O(K log N), query O(1) | Space: O(N * K)
struct k_cover_tree {
  int n, k;
  vector<int> all, cnt;
  vector<vector<int>> curr;

  void recalc(int i) {
    vector<int> base(k + 1);
    base[0] = all[i];
    if (i < n) for (int h = 1; h <= k; ++h) base[h] = curr[i << 1][h]
      + curr[i << 1 | 1][h];
    for (int h = 1; h <= k; ++h) curr[i][h] = base[max<int>(0, h -
      cnt[i])];
  }

  k_cover_tree(int k, vector<int> &x): n(x.size() - 1), k(k), all(2 *
    n), cnt(2 * n), curr(2 * n, vector<int>(k + 1)) {
    for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
    for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1
      | 1];
  }

  void update(int i, int j, int x) {
    int p = i, q = j;
    for (i += n, j += n; i < j; i >>= 1, j >>= 1) {
      if (i & 1) cnt[i] += x, recalc(i++);
      if (j & 1) cnt[--j] += x, recalc(j);
    }
    i = p + n;
    while (i >>= 1) recalc(i);
    j = q + n - 1;
    while (j >>= 1) recalc(j);
  }

  int query() {
    return curr[1][k];
  }
};
```

coprime-set

custom-sets/

```
// Coprime Set: counts elements coprime to a given number using
  Inclusion-Exclusion
// Time: insert/remove/query O(2^K * (K + log D)) | Space: O(N * 2^K)
// Note: P must be a vector of DISTINCT prime factors | K = size of
  P, D = unique divisors in map
struct coprime_set {
  map<int, int> n;

  vector<pair<int, int>> divisors(vector<int> &p) {
    int k = p.size();
    vector<pair<int, int>> a; a.reserve(1 << k);
    for (int m = 0; m < (1 << k); ++m) {
      int d = 1, mu = 1;
      for (int i = 0; i < k; ++i) if (m & (1 << i)) {
        d *= p[i], mu *= -1;
      }
      a.push_back({d, mu});
    }
    return a;
  }

  void insert(vector<int> &p) {
    vector<pair<int, int>> a = divisors(p);
    for (auto &[d, mu] : a) {
      n[d]++;
    }
  }

  void remove(vector<int> &p) {
    vector<pair<int, int>> a = divisors(p);
    for (auto &[d, mu] : a) {
      n[d]--;
    }
  }

  int query(vector<int> &p) {
    int t = 0;
    vector<pair<int, int>> a = divisors(p);
    for (auto &[d, mu] : a) {
      t += n[d] * mu;
    }
    return t;
  }
};
```

interval-set

custom-sets/

```
// Interval Set: maintains disjoint intervals with dynamic active
  elements count
// Time: insert/remove O(log N) amortized | contains O(log N) | total
  O(1) | Space: O(N)
// Note: discrete=true for [l, r] (r-l+1) | discrete=false for
  continuous [l, r) (r-l)
struct interval_set {
  int active = 0, off;
  set<pair<int, int>> s;

  interval_set(bool discrete): off(discrete) {}

  void insert(int l, int r) {
    assert(l < r + off);
    auto itr = s.lower_bound({l, -inf});
    if (itr != s.begin() and prev(itr)->second >= l - off) itr--;
    int nl = l;
    int nr = r;
    while (itr != s.end() and itr->first <= r + off) {
      nl = min(nl, itr->first);
      nr = max(nr, itr->second);
      active -= itr->second - itr->first + off;
      itr = s.erase(itr);
    }
    active += nr - nl + off;
    s.insert({nl, nr});
  }

  void remove(int l, int r) {
    assert(l < r + off);
    auto itr = s.lower_bound({l, -inf});
    if (itr != s.begin() and prev(itr)->second >= l) itr--;
    while (itr != s.end() and itr->first <= r) {
      int cl = itr->first;
      int cr = itr->second;
      active -= cr - cl + off;
      itr = s.erase(itr);
      if (cl < l) {
        s.insert({cl, l - off});
        active += l - cl;
      }
      if (cr > r) {
        s.insert({r + off, cr});
        active += cr - r;
      }
    }
  }

  bool contains(int l, int r) {
    assert(l < r + off);
    auto itr = s.upper_bound({l, inf});
    if (itr == s.begin()) return false;
    itr--;
    return itr->first <= l and r <= itr->second;
  }

  int total() {
    return active;
  }
};
```

range-set

custom-sets/

```
// Range Set: bitset optimized for range updates and queries
// Time: O(N/64) per operation | Space: O(N/64)
// Note: uint = uint64_t (unsigned long long)
struct range_set {
  int n;
  vector<uint> data;

  range_set(int n): n(n), data((n + 63) / 64) {}

  uint from(int i) {
    return ~0ULL << (i & 63);
  }

  uint until(int i) {
```

```
    return ~0ULL >> (63 - (i & 63));
  }

  void apply(int k, bool on, uint mask) {
    if (on) data[k] |= mask;
    else data[k] &= ~mask;
  }

  void update(int l, int r, bool on) {
    int i = l >> 6, j = r >> 6;
    if (i == j) {
      apply(i, on, from(l) & until(r));
    } else {
      apply(i, on, from(l));
      for (int k = i + 1; k < j; ++k) apply(k, on, ~0ULL);
      apply(j, on, until(r));
    }
  }

  int count(int l, int r) {
    int c = 0;
    int i = l >> 6, j = r >> 6;
    if (i == j) {
      c = __builtin_popcountll(data[i] & (from(l) & until(r)));
    } else {
      c += __builtin_popcountll(data[i] & from(l));
      for (int k = i + 1; k < j; ++k) c +=
        __builtin_popcountll(data[k]);
      c += __builtin_popcountll(data[j] & until(r));
    }
    return c;
  }

  bool empty() {
    for (uint mask : data) if (mask) return false;
    return true;
  }

  int prev(int i) {
    for (int k = i >> 6; k >= 0; --k, i = 63) {
      uint mask = data[k] & until(i);
      if (mask) return (k << 6) + 63 - __builtin_clzll(mask);
    }
    return n;
  }

  int next(int i) {
    for (int k = i >> 6; k < data.size(); ++k, i = 0) {
      uint mask = data[k] & from(i);
      if (mask) return (k << 6) + __builtin_ctzll(mask);
    }
    return n;
  }

  int segments() {
    int s = 0;
    for (int k = 0; k < data.size(); ++k) {
      s += __builtin_popcountll((data[k] >> 1) & ~data[k]);
      if ((data[k] & 1) and (k == 0 or (data[k - 1] >> 63) == 0)) s
        += 1;
    }
    return s;
  }
};
```

venice-set

custom-sets/

```
// Venice Set: multiset with global value updates
// Time: O(log N) | Space: O(N)
struct venice_set {
  int k = 0;
  multiset<int> s;

  void insert(int x) {
    s.insert(x - k);
  }

  void remove(int x) {
    if (auto i = s.find(x - k); i != s.end()) s.erase(i);
  }

  void update(int x) {
    k += x;
  }

  int min() {
    return *s.begin() + k;
  }
};
```

disjoint-set-union

disjoint-set-union/

```
struct dsu {
  vector<int> p, s;

  dsu(int n): p(n), s(n, 1) {
    iota(p.begin(), p.end(), 0);
  }

  int find(int a) {
    if (p[a] == a) return a;
    return p[a] = find(p[a]);
  }

  void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return;
    if (s[a] < s[b]) swap(a, b);
    p[b] = a;
    s[a] += s[b];
  }
};

struct dsu_2d {
  int n, m;
  dsu d;

  dsu_2d(int n, int m): n(n), m(m), d(n * m) {}

  int id(int r, int c) {
    return r * m + c;
  }

  int find(int r, int c) {
    return d.find(id(r, c));
  }

  void unite(int r1, int c1, int r2, int c2) {
    d.unite(id(r1, c1), id(r2, c2));
  }
};
```

dynamic-connectivity

disjoint-set-union/

```
struct dynamic_connectivity {
private:
  int n, q;
  vector<vector<pair<int, int>>> tree;
  rb_dsu dsu;

  void insert(pair<int, int> e, int ql, int qr, int p, int l, int r) {
    if (qr < l or ql > r) return;
    if (ql <= l and qr >= r) return void(tree[p].push_back(e));
    int m = (l + r) / 2;
    insert(e, ql, qr, 2 * p, l, m);
    insert(e, ql, qr, 2 * p + 1, m + 1, r);
  }

  void compute(vector<int> &ans, int p, int l, int r) {
    for (auto [a, b] : tree[p]) dsu.unite(a, b);
    if (l == r) {
      ans[l] = dsu.c;
    } else {
      int m = (l + r) / 2;
      compute(ans, 2 * p, l, m);
      compute(ans, 2 * p + 1, m + 1, r);
    }
    for (auto [a, b] : tree[p]) dsu.rollback();
  }

public:
  dynamic_connectivity(int n, int q): n(n), q(q), tree(4 * q), dsu(n)
    {}

  void insert(pair<int, int> e, int ql, int qr) {
    insert(e, ql, qr, 1, 0, q - 1);
  }

  vector<int> compute() {
    vector<int> ans(q);
    compute(ans, 1, 0, q - 1);
    return ans;
  }
};
```

rollback-disjoint-set-union

disjoint-set-union/

```
struct rb_dsu {
  int c;
  vector<int> p, s;
  vector<pair<int, int>> h;

  rb_dsu(int n): c(n), p(n), s(n, 1) {
    iota(p.begin(), p.end(), 0);
  }

  int find(int a) {
    while (p[a] != a) a = p[a];
    return a;
  }

  void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) {
      h.push_back({-1, -1});
    } else {
      if (s[a] < s[b]) swap(a, b);
      h.push_back({a, b});
      c--;
      p[b] = a;
      s[a] += s[b];
    }
  }

  void rollback() {
    assert(h.size());
    auto [a, b] = h.back(); h.pop_back();
    if (a != -1) {
      c++;
      p[b] = b;
      s[a] -= s[b];
    }
  }

  int state() {
    return h.size();
  }
};
```

dynamic-merge-sort-tree

merge-sort-tree/

```
// Merge Sort Tree: range queries with point updates
// Time: build O(N log^2 N) | Update/Query: O(log^2 N) | Space: O(N
  log N)
// Note: uses PBDS ordered_set | pair<int, int> handles duplicate
  values
struct merge_sort_tree {
  int n;
  vector<int> a;
  vector<ordered_set<pair<int, int>>> tree;

  merge_sort_tree(vector<int> &a): n(a.size()), a(a), tree(2 * n) {
    for (int i = 0; i < n; ++i) {
      for (int j = i + n; j > 0; j >>= 1) {
        tree[j].insert({a[i], i});
      }
    }
  }

  void update(int i, int x) {
    for (int j = i + n; j > 0; j >>= 1) {
      tree[j].erase({a[i], i});
      tree[j].insert({x, i});
    }
    a[i] = x;
  }

  int less_or_equal(int i, int j, int x) {
    int c = 0;
    for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
      if (i & 1) c += tree[i++].order_of_key({x + 1, 0});
      if (j & 1) c += tree[--j].order_of_key({x + 1, 0});
    }
    return c;
  }
};
```

merge-sort-tree

merge-sort-tree/

```
struct merge_sort_tree {
  int n;
  vector<vector<int>> tree;

  merge_sort_tree(vector<int> &a): n(a.size()), tree(2 * n) {
    for (int i = 0; i < n; ++i) tree[i + n] = {a[i]};
    for (int i = n - 1; i > 0; --i) merge(tree[i << 1].begin(),
      tree[i << 1].end(), tree[(i << 1) | 1].begin(), tree[(i << 1) |
      1].end(), back_inserter(tree[i]));
  }

  int less_or_equal(int i, int j, int x) {
    int c = 0;
    for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
      if (i & 1) c += upper_bound(tree[i].begin(), tree[i].end(), x)
        - tree[i].begin(), i++;
      if (j & 1) --j, c += upper_bound(tree[j].begin(),
        tree[j].end(), x) - tree[j].begin();
```

```
    }
    return c;
  }
};
```

hash-table

policy-based-data-structures/

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

struct custom_hash {
  size_t operator()(size_t x) const {
    static const size_t time =
      chrono::steady_clock::now().time_since_epoch().count();
    return (x + 0x9e3779b9) ^ time;
  }
};

template<typename T>
using hash_set = gp_hash_table<T, null_type, custom_hash>;

template<typename K, typename V>
using hash_map = gp_hash_table<K, V, custom_hash>;
```

ordered-set

policy-based-data-structures/

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;
```

iterative-segment-tree

segment-tree/

```
struct segment_tree {
  int n;
  vector<int> tree;

  static int merge(int x, int y) {
    return x + y;
  }

  segment_tree(int s): n(__bit_ceil(s)), tree(2 * n) {}

  segment_tree(vector<int> &a): segment_tree(a.size()) {
    for (int i = 0; i < a.size(); ++i) tree[n + i] = a[i];
    for (int i = n - 1; i > 0; --i) tree[i] = merge(tree[i << 1],
      tree[(i << 1) | 1]);
  }

  void update(int i, int x) {
    i += n;
    tree[i] = x;
    while (i >>= 1) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
  }

  int query(int i, int j) {
    int l = 0, r = 0;
    for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
      if (i & 1) l = merge(l, tree[i++]);
      if (j & 1) r = merge(tree[--j], r);
    }
    return merge(l, r);
  }
};

struct segment_tree_2d {
  int n, m;
  vector<segment_tree> tree;

  segment_tree_2d(int r, int c): n(__bit_ceil(r)), m(__bit_ceil(c)),
    tree(2 * n, segment_tree(m)) {}

  segment_tree_2d(vector<vector<int>> &a): segment_tree_2d(a.size(),
    a[0].size()) {
    for (int i = 0; i < a.size(); ++i) tree[n + i] =
      segment_tree(a[i]);
    for (int i = n - 1; i > 0; --i) {
      for (int j = 1; j < 2 * m; ++j) {
        tree[i].tree[j] = segment_tree::merge(tree[i << 1].tree[j],
          tree[(i << 1) | 1].tree[j]);
      }
    }
  }

  void update(int i, int j, int x) {
    tree[i += n].update(j, x);
    while (i >>= 1) tree[i].update(j, segment_tree::merge(tree[i <<
      1].tree[j + m], tree[(i << 1) | 1].tree[j + m]));
  }

  int query(int i1, int j1, int i2, int j2) {
    int l = 0, r = 0;
    for (i1 += n, i2 += n + 1; i1 < i2; i1 >>= 1, i2 >>= 1) {
      if (i1 & 1) l = segment_tree::merge(l, tree[i1++].query(j1,
        j2));
      if (i2 & 1) r = segment_tree::merge(tree[--i2].query(j1, j2),
        r);
    }
    return segment_tree::merge(l, r);
  }
};
```

segment-tree-beats

segment-tree/

```
// Segment Tree Beats: advanced range updates via condition pruning
// Time: update amortized O(log^2 N), query O(log N) | Space: O(N)
// Note: supports sum, add, chmin, chmax and mod over a range
struct segment_tree_beats {
private:
  struct node {
    int sum, lazy;
    int max1, max2, maxc;
    int min1, min2, minc;
  };

  int n;
  vector<node> tree;

  void push_up(int p) {
    int l = 2 * p, r = 2 * p + 1;
    tree[p].sum = tree[l].sum + tree[r].sum;

    if (tree[l].max1 == tree[r].max1) {
      tree[p].max1 = tree[l].max1;
      tree[p].max2 = max(tree[l].max2, tree[r].max2);
      tree[p].maxc = tree[l].maxc + tree[r].maxc;
    } else if (tree[l].max1 > tree[r].max1) {
      tree[p].max1 = tree[l].max1;
      tree[p].max2 = max(tree[l].max2, tree[r].max1);
      tree[p].maxc = tree[l].maxc;
    } else {
      tree[p].max1 = tree[r].max1;
      tree[p].max2 = max(tree[l].max1, tree[r].max2);
      tree[p].maxc = tree[r].maxc;
    }

    if (tree[l].min1 == tree[r].min1) {
      tree[p].min1 = tree[l].min1;
      tree[p].min2 = min(tree[l].min2, tree[r].min2);
      tree[p].minc = tree[l].minc + tree[r].minc;
    } else if (tree[l].min1 < tree[r].min1) {
      tree[p].min1 = tree[l].min1;
      tree[p].min2 = min(tree[l].min2, tree[r].min1);
      tree[p].minc = tree[l].minc;
    } else {
      tree[p].min1 = tree[r].min1;
      tree[p].min2 = min(tree[l].min1, tree[r].min2);
      tree[p].minc = tree[r].minc;
    }
  }

  void apply_add(int p, int l, int r, int v) {
    tree[p].sum += v * (r - l + 1);
    tree[p].max1 += v;
    if (tree[p].max2 != -inf) tree[p].max2 += v;
    tree[p].min1 += v;
    if (tree[p].min2 != inf) tree[p].min2 += v;
    tree[p].lazy += v;
  }

  void apply_min(int p, int v) {
    if (v >= tree[p].max1) return;
    tree[p].sum -= (tree[p].max1 - v) * tree[p].maxc;
    if (tree[p].min1 == tree[p].max1) tree[p].min1 = v;
    if (tree[p].min2 == tree[p].max1) tree[p].min2 = v;
    tree[p].max1 = v;
  }

  void apply_max(int p, int v) {
    if (v <= tree[p].min1) return;
    tree[p].sum += (v - tree[p].min1) * tree[p].minc;
    if (tree[p].max1 == tree[p].min1) tree[p].max1 = v;
    if (tree[p].max2 == tree[p].min1) tree[p].max2 = v;
    tree[p].min1 = v;
  }

  void push_down(int p, int l, int r) {
    int m = (l + r) / 2;
    int left = 2 * p, right = 2 * p + 1;

    if (tree[p].lazy != 0) {
      apply_add(left, l, m, tree[p].lazy);
      apply_add(right, m + 1, r, tree[p].lazy);
      tree[p].lazy = 0;
    }

    apply_min(left, tree[p].max1);
    apply_min(right, tree[p].max1);

    apply_max(left, tree[p].min1);
    apply_max(right, tree[p].min1);
  }

  void build(const vector<int> &a, int p, int l, int r) {
    tree[p].lazy = 0;
    if (l == r) {
      tree[p].sum = tree[p].max1 = tree[p].min1 = a[l];
      tree[p].maxc = tree[p].minc = 1;
      tree[p].max2 = -inf;
      tree[p].min2 = inf;
    } else {
      int m = (l + r) / 2;
      build(a, 2 * p, l, m);
      build(a, 2 * p + 1, m + 1, r);
      push_up(p);
    }
  }

  void update_add(int ql, int qr, int v, int p, int l, int r) {
    if (ql <= l and r <= qr) {
      apply_add(p, l, r, v);
      return;
    }
    push_down(p, l, r);
    int m = (l + r) / 2;
    if (ql <= m) update_add(ql, qr, v, 2 * p, l, m);
    if (qr > m) update_add(ql, qr, v, 2 * p + 1, m + 1, r);
    push_up(p);
  }

  void update_min(int ql, int qr, int v, int p, int l, int r) {
    if (v >= tree[p].max1) return;
    if (ql <= l and r <= qr and v > tree[p].max2) {
      apply_min(p, v);
      return;
    }
    push_down(p, l, r);
    int m = (l + r) / 2;
    if (ql <= m) update_min(ql, qr, v, 2 * p, l, m);
    if (qr > m) update_min(ql, qr, v, 2 * p + 1, m + 1, r);
    push_up(p);
  }

  void update_max(int ql, int qr, int v, int p, int l, int r) {
    if (v <= tree[p].min1) return;
    if (ql <= l and r <= qr and v < tree[p].min2) {
      apply_max(p, v);
      return;
    }
    push_down(p, l, r);
    int m = (l + r) / 2;
    if (ql <= m) update_max(ql, qr, v, 2 * p, l, m);
    if (qr > m) update_max(ql, qr, v, 2 * p + 1, m + 1, r);
    push_up(p);
  }

  void update_mod(int ql, int qr, int v, int p, int l, int r) {
    if (ql <= l and r <= qr) {
      if (tree[p].max1 < v) return;
      if (tree[p].min1 == tree[p].max1) {
        apply_add(p, l, r, (tree[p].max1 % v) - tree[p].max1);
        return;
      }
    }
    push_down(p, l, r);
    int m = (l + r) / 2;
    if (ql <= m) update_mod(ql, qr, v, 2 * p, l, m);
    if (qr > m) update_mod(ql, qr, v, 2 * p + 1, m + 1, r);
    push_up(p);
  }

  int query_sum(int ql, int qr, int p, int l, int r) {
    if (ql <= l and r <= qr) return tree[p].sum;
    push_down(p, l, r);
    int m = (l + r) / 2;
    int ans = 0;
    if (ql <= m) ans += query_sum(ql, qr, 2 * p, l, m);
    if (qr > m) ans += query_sum(ql, qr, 2 * p + 1, m + 1, r);
    return ans;
  }

public:
  segment_tree_beats(int n) : n(n), tree(4 * n) {}
```

```
  segment_tree_beats(const vector<int> &a) : n(a.size()), tree(4 * n)
    {
    build(a, 1, 0, n - 1);
  }

  void update_add(int l, int r, int v) {
    update_add(l, r, v, 1, 0, n - 1);
  }

  void update_min(int l, int r, int v) {
    update_min(l, r, v, 1, 0, n - 1);
  }

  void update_max(int l, int r, int v) {
    update_max(l, r, v, 1, 0, n - 1);
  }

  void update_mod(int l, int r, int v) {
    update_mod(l, r, v, 1, 0, n - 1);
  }

  int query_sum(int l, int r) {
    return query_sum(l, r, 1, 0, n - 1);
  }
};
```

segment-tree

segment-tree/

```
struct segment_tree {
  struct node {
    int sum = 0, lazy = 0;
    int l = 0, r = 0;
  };

  void merge(node &c, const node &a, const node &b) {
    c.sum = a.sum + b.sum;
  }

  void apply(node &a, int x, int len) {
    a.sum += x * len;
    a.lazy += x;
  }

  int low, high, root = 0;
  bool keep;
  vector<node> tree = {node()};

  segment_tree(int low, int high, bool keep): low(low), high(high),
    keep(keep) {}

  int touch(int p) {
    if (p and not keep) return p;
    tree.push_back(tree[p]);
    return tree.size() - 1;
  }

  void push(int p, int l, int r) {
    if (tree[p].lazy == tree[0].lazy) return;
    int m = l + (r - l) / 2;
    tree[p].l = touch(tree[p].l), tree[p].r = touch(tree[p].r);
    apply(tree[tree[p].l], tree[p].lazy, m - l + 1);
    apply(tree[tree[p].r], tree[p].lazy, r - m);
    tree[p].lazy = tree[0].lazy;
  }

  int set(int i, const node &x, int p = -1, int l = 1, int r = 0) {
    if (l > r) return root = set(i, x, p < 0 ? root : p, low, high);
    p = touch(p);
    if (l == r) return tree[p] = x, p;
    push(p, l, r);
    int m = l + (r - l) / 2;
    if (i <= m) tree[p].l = set(i, x, tree[p].l, l, m);
    else tree[p].r = set(i, x, tree[p].r, m + 1, r);
    merge(tree[p], tree[tree[p].l], tree[tree[p].r]);
    return p;
  }

  int update(int ql, int qr, int x, int p = -1, int l = 1, int r = 0)
    {
    if (l > r) return root = update(ql, qr, x, p < 0 ? root : p, low,
      high);
    if (qr < l or r < ql) return p;
    p = touch(p);
    if (ql <= l and r <= qr) return apply(tree[p], x, r - l + 1), p;
    push(p, l, r);
    int m = l + (r - l) / 2;
    tree[p].l = update(ql, qr, x, tree[p].l, l, m);
    tree[p].r = update(ql, qr, x, tree[p].r, m + 1, r);
    merge(tree[p], tree[tree[p].l], tree[tree[p].r]);
    return p;
  }

  node query(int ql, int qr, int p = -1, int l = 1, int r = 0) {
    if (l > r) return query(ql, qr, p < 0 ? root : p, low, high);
    if (qr < l or r < ql) return node();
    if (not p or (ql <= l and r <= qr)) return tree[p];
    push(p, l, r);
    int m = l + (r - l) / 2;
    node c;
    merge(c, query(ql, qr, tree[p].l, l, m), query(ql, qr, tree[p].r,
      m + 1, r));
    return c;
  }
};
```

sparse-table

sparse-table/

```
struct sparse_table {
  int n, l;
  vector<vector<int>> table;

  static int merge(int x, int y) {
    return gcd(x, y);
  }

  sparse_table(vector<int> &a): n(a.size()), l(__lg(n)), table(l + 1,
    vector<int>(n)) {
    for (int i = 0; i < n; ++i) table[0][i] = a[i];
    for (int k = 1; k <= l; ++k) {
      for (int i = 0; i + (1 << k) <= n; ++i) {
        table[k][i] = merge(table[k - 1][i], table[k - 1][i + (1 <<
          (k - 1))]);
      }
    }
  }

  int query(int i, int j) {
    int k = __lg(j - i + 1);
    return merge(table[k][i], table[k][j - (1 << k) + 1]);
  }
};

struct sparse_table_2d {
  int n, m, l;
  vector<vector<sparse_table>> table;

  sparse_table_2d(vector<vector<int>> &a): n(a.size()),
    m(a[0].size()), l(__lg(n)), table(l + 1) {
    for (int i = 0; i < n; ++i) {
      table[0].push_back(sparse_table(a[i]));
    }
    for (int k = 1; k <= l; ++k) {
      for (int i = 0; i + (1 << k) <= n; ++i) {
        vector<int> h(m);
        for (int j = 0; j < m; ++j) {
          int x = table[k - 1][i].table[0][j];
          int y = table[k - 1][i + (1 << (k - 1))].table[0][j];
          h[j] = sparse_table::merge(x, y);
        }
        table[k].push_back(sparse_table(h));
      }
    }
  }

  int query(int i1, int j1, int i2, int j2) {
    int k = __lg(i2 - i1 + 1);
    int x = table[k][i1].query(j1, j2);
    int y = table[k][i2 - (1 << k) + 1].query(j1, j2);
    return sparse_table::merge(x, y);
  }
};
```

implicit-treap

treap/

```
// Implicit Treap: dynamic array with range updates and queries
// Time: operations O(log N) average | Space: O(N)
// Note: supports range modifications | uses lazy propagation and
  implicit indices
struct treap {
  struct node {
    int x, p, l = 0, r = 0;
    int sum, size = 1, lazy = 0, rev = 0;

    node(int x, int p = rng()): x(x), p(p), sum(x) {}
  };

  int root = 0;
  vector<node> t;

  treap(int n = 0) {
    t.reserve(n + 1);
    t.push_back(node(0));
    t[0].size = 0;
  }

  void apply(int i, int x) {
    if (i == 0) return;
    t[i].x += x;
    t[i].sum += x * t[i].size;
    t[i].lazy += x;
  }

  void flip(int i) {
    if (i == 0) return;
    swap(t[i].l, t[i].r);
    t[i].rev ^= 1;
  }

  void push(int i) {
    if (i == 0) return;
    if (t[i].rev != 0) {
      flip(t[i].l);
      flip(t[i].r);
      t[i].rev = 0;
    }
    if (t[i].lazy != 0) {
      apply(t[i].l, t[i].lazy);
      apply(t[i].r, t[i].lazy);
      t[i].lazy = 0;
    }
  }

  void pull(int i) {
    if (i == 0) return;
    t[i].size = t[t[i].l].size + 1 + t[t[i].r].size;
    t[i].sum = t[t[i].l].sum + t[i].x + t[t[i].r].sum;
  }

  pair<int, int> split(int i, int k) {
    if (i == 0) return {0, 0};
    push(i);
    int s = t[t[i].l].size;
    if (s >= k) {
      auto [l, r] = split(t[i].l, k);
      t[i].l = r;
      pull(i);
      return {l, i};
    } else {
      auto [l, r] = split(t[i].r, k - s - 1);
      t[i].r = l;
      pull(i);
      return {i, r};
    }
  }

  int merge(int l, int r) {
    if (l == 0 or r == 0) return l | r;
    if (t[l].p > t[r].p) {
      push(l);
      t[l].r = merge(t[l].r, r);
      pull(l);
      return l;
    } else {
      push(r);
      t[r].l = merge(l, t[r].l);
      pull(r);
      return r;
    }
  }

  void insert(int k, int x) {
    auto [l, r] = split(root, k);
    int i = t.size();
    t.push_back(node(x));
    root = merge(merge(l, i), r);
  }

  void erase(int k) {
    auto [l, m] = split(root, k);
    auto [_, r] = split(m, 1);
    root = merge(l, r);
  }

  void move(int i, int j, int k) {
    auto [h, q] = split(root, j + 1);
    auto [p, m] = split(h, i);
    auto [l, r] = split(merge(p, q), k);
    root = merge(merge(l, m), r);
  }

  void reverse(int i, int j) {
    auto [h, r] = split(root, j + 1);
    auto [l, m] = split(h, i);
    flip(m);
    root = merge(merge(l, m), r);
  }
```

```
  void update(int i, int j, int x) {
    auto [h, r] = split(root, j + 1);
    auto [l, m] = split(h, i);
    apply(m, x);
    root = merge(merge(l, m), r);
  }

  int query(int i, int j) {
    auto [h, r] = split(root, j + 1);
    auto [l, m] = split(h, i);
    int q = t[m].sum;
    root = merge(merge(l, m), r);
    return q;
  }

  int kth(int k) {
    int i = root;
    while (i != 0) {
      push(i);
      int s = t[t[i].l].size;
      if (k < s) {
        i = t[i].l;
      } else if (k > s) {
        k -= s + 1;
        i = t[i].r;
      } else {
        return t[i].x;
      }
    }
    return -1;
  }
};
```

treap

treap/

```
// Treap: randomized binary search tree (split/merge)
// Time: operations O(log N) average | Space: O(N)
// Note: combines bst and heap properties | requires mt19937 for
  priorities
struct treap {
  struct node {
    int x, p, l = 0, r = 0, s = 1;

    node(int x, int p = rng()): x(x), p(p) {}
  };

  int root = 0;
  vector<node> t;

  treap(int n = 0) {
    t.reserve(n + 1);
    t.push_back(node(0));
    t[0].s = 0;
  }

  void pull(int i) {
    if (i == 0) return;
    t[i].s = 1 + t[t[i].l].s + t[t[i].r].s;
  }

  pair<int, int> split(int i, int x) {
    if (i == 0) return {0, 0};
    if (t[i].x <= x) {
      auto [l, r] = split(t[i].r, x);
      t[i].r = l;
      pull(i);
      return {i, r};
    } else {
      auto [l, r] = split(t[i].l, x);
      t[i].l = r;
      pull(i);
      return {l, i};
    }
  }

  int merge(int l, int r) {
    if (l == 0 or r == 0) return l | r;
    if (t[l].p > t[r].p) {
      t[l].r = merge(t[l].r, r);
      pull(l);
      return l;
    } else {
      t[r].l = merge(l, t[r].l);
      pull(r);
      return r;
    }
  }

  void insert(int x) {
    auto [l, r] = split(root, x);
    int i = t.size();
    t.push_back(node(x));
    root = merge(merge(l, i), r);
  }

  void erase(int x) {
    auto [l, h] = split(root, x - 1);
    auto [m, r] = split(h, x);
    m = merge(t[m].l, t[m].r);
    root = merge(merge(l, m), r);
  }

  int index(int x) {
    int i = root, k = 0;
    while (i != 0) {
      if (t[i].x < x) k += 1 + t[t[i].l].s, i = t[i].r;
      else i = t[i].l;
    }
    return k;
  }

  int kth(int k) {
    int i = root;
    while (i != 0) {
      int s = t[t[i].l].s;
      if (k < s) i = t[i].l;
      else if (k > s) k -= 1 + s, i = t[i].r;
      else return t[i].x;
    }
    return -1;
  }
};
```

wavelet-matrix

```
// Wavelet Matrix: range quantile, range frequency, k-th smallest
// Time: build O(N log V) | Queries O(log V) | Space: O(N log V)
// Note: coordinate compression is REQUIRED before building.
struct wavelet_matrix {
  int n, log;
  vector<vector<int>> pref;
  vector<int> mid;

  wavelet_matrix(vector<int> a): n(a.size()),
    log(__lg(*max_element(a.begin(), a.end()) | 1)), pref(log + 1,
    vector<int>(n + 1)), mid(log + 1) {
    for (int b = log; b >= 0; --b) {
      for (int i = 0; i < n; ++i) pref[b][i + 1] = pref[b][i] +
        (((a[i] >> b) & 1) == 0);
      mid[b] = stable_partition(a.begin(), a.end(), [&](int x)
        {return ((x >> b) & 1) == 0;}) - a.begin();
    }
  }

  int kth(int i, int j, int k) {
    int x = 0;
    for (int b = log; b >= 0; --b) {
      int l = pref[b][i], r = pref[b][j + 1], z = r - l;
      if (k < z) i = l, j = r - 1;
      else x |= 1 << b, i += mid[b] - l, j += mid[b] - r, k -= z;
    }
    return x;
  }

  int less_or_equal(int i, int j, int x) {
    int c = 0;
    if (x >= (1 << (log + 1))) x = (1 << (log + 1)) - 1;
    for (int b = log; b >= 0; --b) {
      int l = pref[b][i], r = pref[b][j + 1], z = r - l;
      if (((x >> b) & 1) == 0) i = l, j = r - 1;
      else c += z, i += mid[b] - l, j += mid[b] - r;
    }
    if (i <= j) c += j - i + 1;
    return c;
  }
};
```

theory

burnside-lemma

Lema de Burnside

O Lema de Burnside é utilizado em combinatória matemática para contar o número de objetos estruturalmente únicos (classes de equivalência) em um conjunto, eliminando duplicatas geradas por simetrias (como rotações ou reflexões).

Fórmula Geral:

$$|X/G| = \frac{1}{|G|} \sum_{\pi \in G} |X^\pi|$$

Onde:

- $|X/G|$: Número de objetos únicos.
- $|G|$: Tamanho do grupo de simetrias (total de permutações).
- $|X^\pi|$: Número de pontos fixos (representações que não mudam ao aplicar a permutação π).

Aplicação Clássica: Problema do Colar (Necklace)

Conta o número de formas únicas de montar um colar de tamanho n com m cores possíveis para as pérolas, considerando que colares rotacionados são iguais.

Para uma rotação de tamanho i em um colar de tamanho n, a permutação se decompõe em $\gcd(n, i)$ ciclos. Para que a coloração seja um ponto fixo sob essa rotação, todas as pérolas de um mesmo ciclo devem ter a mesma cor. Como temos m cores, existem $m^{\gcd(n,i)}$ colorações invariantes (pontos fixos) para essa rotação.

Fórmula para Rotações em O(N):

$$\text{Colares} = \frac{1}{n} \sum_{i=1}^{n} m^{\gcd(n,i)} \pmod{10^9 + 7}$$

exponent-reduction-euler-theorem

Redução de Expoentes Grandes (Teorema Estendido de Euler)

Para calcular $a^e \pmod m$ quando o expoente e é extremamente grande (como em strings com 10^5 dígitos ou torres de potências a^{b^c}), utiliza-se a seguinte identidade fundamental:

$$a^e \equiv \begin{cases} a^e \pmod m, & \text{se } e < \varphi(m) \\ a^{(e \bmod \varphi(m)) + \varphi(m)} \pmod m, & \text{se } e \geq \varphi(m) \end{cases}$$

Resumo de Aplicação

- **Quando se aplica:** Funciona universalmente para **qualquer** par de inteiros a e m, mesmo quando a e m não são primos entre si ($\gcd(a, m) \neq 1$).
- **Caso $e < \varphi(m)$:** Não reduza o expoente. Reduzi-lo antes de atingir $\varphi(m)$ altera o resultado quando $\gcd(a, m) > 1$.
- **Caso $e \geq \varphi(m)$:** Substitua o expoente e por $(e \bmod \varphi(m)) + \varphi(m)$. A adição do termo $+\varphi(m)$ garante que a fase transitória da potenciação seja respeitada.

flow-bipartite-graphs-reductions

Fluxo, Grafos Bipartidos e Reduções

Equivalências em Grafos Bipartidos

Dado um grafo bipartido $G = (L \cup R, E)$ e o Pareamento Máximo M (obtido via Fluxo Máximo $S \to L \to R \to T$ com capacidades 1):

- **Max Bipartite Matching (MBM):** Valor do fluxo máximo $|f_{\max}|$.
- **Min Vertex Cover (MVC):** Mínimo de vértices para cobrir todas as arestas.

$$|\text{MVC}| = |\text{MBM}| \quad \text{(Teorema de König)}$$

- **Max Independent Set (MIS):** Máximo de vértices sem arestas entre si.

$$|\text{MIS}| = |V| - |\text{MVC}| = |V| - |\text{MBM}|$$

- **Min Path Cover (DAG):** Menor número de caminhos vértices-disjuntos para cobrir um DAG. Duplique cada vértice $u \to (u_L, u_R)$. Para cada aresta $u \to v$, adicione (u_L, v_R) com capacidade 1.

$$\text{Quantidade de Caminhos} = |V| - |\text{MBM}|$$

- **Teorema de Dilworth (Anticadeias em DAGs):** Uma anticadeia é o maior conjunto de vértices onde nenhum alcança o outro.

$$\text{Tamanho da Maior Anticadeia} = \text{Quantidade Mínima de Caminhos}$$

Teoremas em Grafos Gerais (Não Bipartidos)

Enquanto o Teorema de König falha fora de grafos bipartidos, as **Identidades de Gallai** valem para **qualquer grafo** $G = (V, E)$ sem vértices isolados:

- **Identidades de Gallai:**

$$|\text{MIS}| + |\text{MVC}| = |V|$$

$$|\text{Max Matching}| + |\text{Min Edge Cover}| = |V|$$

- **Min Edge Cover (Cobertura Mínima de Arestas):** Menor número de arestas necessárias para cobrir todos os vértices.

$$|\text{Min Edge Cover}| = |V| - |\text{Max Matching}|$$

Como obter: Encontre o Pareamento Máximo. Para cada vértice não pareado, adicione qualquer aresta incidente a ele.
- **Alertas para Programação Competitiva:**
  - **König não vale:** |MVC| ≠ |Max Matching| em grafos gerais (ex: no triângulo C_3, |MVC| = 2 e |Matching| = 1).
  - **Complexidade:** MVC e MIS tornam-se problemas **NP-Hard** em grafos gerais.
  - **Fluxo não resolve Pareamento Geral:** Para pareamento em grafos arbitrários (com ciclos ímpares), o algoritmo de fluxo falha. É necessário usar o **Algoritmo de Blossom (Edmonds)**.

Como Recuperar a Resposta (Reconstrução)

Passo Inicial (Grafo Residual G_f): Rode uma DFS ou BFS a partir da fonte S considerando apenas arestas com **capacidade residual** > 0.

Guarde os vértices visitados no conjunto S_{reach}.

- **Pareamento Máximo (MBM):**
  - Uma aresta original (u, v) com $u \in L$ e $v \in R$ pertence ao pareamento se tiver **fluxo positivo**.
- **Min Cut (Arestas do Corte Mínimo):**
  - O corte divide o grafo em visitados (S_{reach}) e não visitados (T_{reach}).
  - Uma aresta (u, v) do **grafo original** pertence ao corte se u foi visitado e v não foi visitado:

$$u \in S_{\text{reach}} \quad \text{e} \quad v \notin S_{\text{reach}}$$

  - Essas arestas estarão obrigatoriamente **saturadas** (fluxo = capacidade).
- **Min Vertex Cover (MVC em Bipartidos):**
  - Pegue os vértices do lado esquerdo L **NÃO visitados** pela DFS.
  - Pegue os vértices do lado direito R **VISITADOS** pela DFS.

$$\text{MVC} = (L \setminus S_{\text{reach}}) \cup (R \cap S_{\text{reach}})$$

- **Max Independent Set (MIS em Bipartidos):**
  - É o **complemento** do Min Vertex Cover ($V \setminus$ MVC):

$$\text{MIS} = (L \cap S_{\text{reach}}) \cup (R \setminus S_{\text{reach}})$$

  - Selecione L **VISITADOS** e R **NÃO visitados**.
- **Min Path Cover (Recuperar Caminhos no DAG):**
  - Reconstrua as cadeias seguindo as arestas (u_L, v_R) que possuem fluxo = 1.
  - Todo vértice u_L que **não** tem fluxo saindo representa o **fim** de um caminho.

Reduções Clássicas de Fluxo

- **Project Selection (Maximum Weight Closure):** Queremos escolher um subconjunto de itens para maximizar o ganho líquido. Se um item u for escolhido e depender de v ($u \to v$), o item v obrigatoriamente precisa ser escolhido junto.
  - Item com **lucro** $p_u > 0$: aresta $S \to u$ com capacidade p_u.
  - Item com **custo** $c_u > 0$: aresta $u \to T$ com capacidade c_u.
  - **Dependência** $u \to v$: aresta $u \to v$ com capacidade ∞.
  - **Corte Mínimo:** Representa a perda mínima.
  - **Lucro Máximo:** $\sum$(Lucros Positivos) − Min Cut.
  - **Itens Selecionados:** Conjunto $S_{\text{reach}} \setminus \{S\}$.
- **Capacidade em Vértices:** Split no vértice $u \to (u_{\text{in}}, u_{\text{out}})$. Adicione aresta $u_{\text{in}} \to u_{\text{out}}$ com cap $= c(u)$. Arestas que entram em u vão para u_{in}, e as que saem vão de u_{out}.

Fluxo com Limite Inferior nas Arestas (Circulation with Demands)

Use essa técnica quando uma aresta $u \to v$ possuir uma capacidade mínima obrigatória L e uma máxima R ($L \le f \le R$). O algoritmo tradicional não entende limites inferiores, então precisamos usar vértices fictícios para forçar esse fluxo.

Construção da Rede:

- Crie dois vértices auxiliares: uma **Super Fonte** (SS) e um **Super Sumidouro** (TT).
- Para cada aresta original $u \to v$ com limite $[L, R]$, adicione na nova rede uma aresta $u \to v$ com **capacidade** $R - L$.
- Mantenha um balanço para cada vértice v. Seja $in[v]$ a soma de todos os limites L das arestas que **entram** em v, e $out[v]$ a soma de todos os limites L das arestas que **saem** de v.
- Adicione uma aresta $SS \to v$ com capacidade $in[v]$.
- Adicione uma aresta $v \to TT$ com capacidade $out[v]$.
- Adicione uma aresta do sumidouro original para a fonte original ($T \to S$) com capacidade ∞ (para permitir a circulação).

Verificação e Resposta:

- Rode o Fluxo Máximo partindo de SS para TT.
- **Existe solução válida?** Apenas se **todas** as arestas saindo de SS (ou chegando em TT) estiverem totalmente saturadas. Ou seja, se o Max Flow $= \sum in[v]$.
- **Recuperar fluxo nas arestas:** O fluxo real em uma aresta original $u \to v$ será: $L +$ (fluxo que passou pela aresta $u \to v$ na nova rede).
- **Para achar o Max Flow real de S a T:** Após verificar que o fluxo inicial é válido, remova os vértices SS e TT e suas arestas, remova a aresta infinita $T \to S$, e rode mais uma vez o Fluxo Máximo de S para T na rede residual.

johnson-algorithm-potential-method

O Algoritmo de Johnson e o Método dos Potenciais

Na programação competitiva, o problema de Caminhos Mais Curtos entre Todos os Pares (APSP - *All-Pairs Shortest Paths*) é frequentemente resolvido com o algoritmo de Floyd-Warshall em $\mathcal{O}(V^3)$. No entanto, para grafos esparsos, essa complexidade pode ser um gargalo. O Algoritmo de Johnson resolve o APSP em $\mathcal{O}(VE \log V)$, o que é muito mais eficiente em grafos onde $E \ll V^2$, mesmo na presença de arestas com pesos negativos.

O Método dos Potenciais

Se o grafo tiver apenas pesos não-negativos, poderíamos simplesmente executar o algoritmo de Dijkstra a partir de cada vértice, obtendo $\mathcal{O}(VE \log V)$. O problema surge com arestas negativas, que quebram a premissa gulosa do Dijkstra.

Para contornar isso, usamos o **Método dos Potenciais**. A ideia é atribuir um valor (potencial) $h(u)$ a cada vértice $u \in V$ e reponderar cada aresta (u, v) de peso $w(u, v)$ com um novo peso $w'(u, v)$ definido como:

$$w'(u, v) = w(u, v) + h(u) - h(v)$$

Essa reponderação tem duas propriedades matemáticas fundamentais:

1. **Preservação dos Caminhos Mais Curtos:** Para qualquer caminho $p = v_0 \to v_1 \to \cdots \to v_k$, o novo peso total do caminho é:

$$w'(p) = w(p) + h(v_0) - h(v_k)$$

   Como o termo $h(v_0) - h(v_k)$ depende apenas dos vértices de início e fim, todos os caminhos entre v_0 e v_k sofrem exatamente a mesma alteração. Logo, o caminho mais curto no grafo original continua sendo o caminho mais curto no grafo reponderado.
2. **Ciclos Negativos:** O peso de um ciclo não é alterado pela reponderação, pois o vértice de início e fim é o mesmo, o que faz com que $h(v_0) - h(v_k) = 0$.

O que torna um Potencial Válido?

Nosso objetivo ao alterar os pesos é habilitar o uso do algoritmo de Dijkstra, que exige que todas as arestas possuam peso não-negativo para garantir a sua corretude. Assim, um potencial h é considerado **válido** se, e somente se, para toda aresta $(u, v) \in E$, o novo peso não for negativo:

$$w'(u, v) \ge 0$$

Substituindo a definição de w', isso significa que:

$$w(u, v) + h(u) - h(v) \ge 0$$

Isolando os potenciais, chegamos à restrição fundamental de um potencial válido:

$$h(v) - h(u) \le w(u, v)$$

Essa inequação tem exatamente o mesmo formato da desigualdade triangular que rege os caminhos mais curtos em grafos.

Isso também nos prova matematicamente por que é impossível resolver o problema se houver um **ciclo de peso negativo**. Imagine um ciclo $C = v_1 \to v_2 \to \cdots \to v_k \to v_1$. Se tentarmos encontrar um potencial válido e somarmos a inequação $h(v) - h(u) \le w(u, v)$ para todas as arestas desse ciclo, o lado esquerdo será exatamente 0 (pois todo vértice aparece uma vez subtraindo e uma vez somando). O lado direito será a soma dos pesos do ciclo, $w(C)$. Teríamos então:

$$0 \le w(C)$$

Se $w(C)$ for um valor negativo, temos um absurdo matemático ($0 \le$ negativo). Isso prova de forma elegante que, se o grafo possui um ciclo de peso negativo, é matematicamente impossível formular uma função potencial válida.

O Algoritmo de Johnson

O objetivo de Johnson é encontrar essa função potencial h válida. Se garantirmos isso, podemos rodar o Dijkstra em segurança a partir de qualquer vértice.

Os passos do algoritmo são:

1. **Vértice Virtual:** Adicione um novo vértice s ao grafo, com arestas de peso 0 direcionadas para todos os outros vértices do grafo original.
2. **Bellman-Ford:** Execute o algoritmo de Bellman-Ford a partir do vértice s.
   - Se o Bellman-Ford detectar um ciclo de peso negativo, o algoritmo para, pois o problema de caminho mais curto não tem solução.
   - Caso contrário, defina o potencial de cada vértice u como a distância mínima de s até u, ou seja, $h(u) = \delta(s, u)$.
3. **Reponderação:** Pela desigualdade triangular do caminho mais curto, sabemos que $\delta(s, v) \le \delta(s, u) + w(u, v)$. Substituindo por h, temos $h(v) \le h(u) + w(u, v)$. Rearranjando a inequação, obtemos:

$$w(u, v) + h(u) - h(v) \ge 0$$

   Logo, definimos $w'(u, v) = w(u, v) + h(u) - h(v)$ e agora garantimos que todas as arestas são não-negativas.
4. **Dijkstra Múltiplo:** Remova o vértice virtual s. Para cada vértice $u \in V$, execute o algoritmo de Dijkstra no grafo reponderado para encontrar as menores distâncias $d'(u, v)$.
5. **Reajuste das Distâncias:** A distância real no grafo original é recuperada revertendo a fórmula do potencial:

$$d(u, v) = d'(u, v) - h(u) + h(v)$$

Na prática das maratonas de programação, o Algoritmo de Johnson é a ferramenta ideal quando você se depara com um grafo grande e esparso, contendo arestas de peso negativo, e que exigiria múltiplas consultas, situação em que o algoritmo de Floyd-Warshall resultaria inevitavelmente em *Time Limit Exceeded* (TLE).

manhattan-to-chebyshev

Manhattan para Chebyshev

Na programação competitiva, problemas que envolvem distâncias em um grid (plano 2D) frequentemente utilizam a **Distância de Manhattan**. A distância de Manhattan entre dois pontos $A = (x_1, y_1)$ e $B = (x_2, y_2)$ é calculada pela soma das diferenças absolutas de suas coordenadas:

$$d_M(A, B) = |x_1 - x_2| + |y_1 - y_2|$$

A Dificuldade da Distância de Manhattan

Matematicamente, lidar com a soma de dois valores absolutos pode ser desconfortável e custoso. Como as coordenadas x e y estão "amarradas" na mesma equação pela soma, não podemos analisá-las ou otimizá-las separadamente.

Se um problema pede para encontrarmos a maior distância entre N pontos, testar todos os pares possíveis exigiria um tempo da ordem de $O(N^2)$, o que é muito lento e frequentemente resulta em *Time Limit Exceeded* (TLE) em competições. Precisamos de uma forma de separar x e y.

A Transformação: Rotacionando o Plano

Para desvincular as coordenadas, utilizamos um truque geométrico elegante: rotacionar o sistema de coordenadas em 45 graus. Na prática, pegamos cada ponto original (x, y) e o convertemos em um novo ponto (u, v) aplicando as seguintes equações:

$$u = x + y$$

$$v = x - y$$

Neste novo plano de coordenadas (u, v), medimos a distância usando a **Distância de Chebyshev**. Em vez de somar as diferenças, a distância de Chebyshev pega o valor máximo entre as diferenças dos eixos:

$$d_C(A', B') = \max(|u_1 - u_2|, |v_1 - v_2|)$$

A grande "mágica" dessa transformação é a garantia de que a distância de Manhattan no plano original é **exatamente igual** à distância de Chebyshev no plano transformado:

$$d_M(A, B) = d_C(A', B')$$

A Intuição: Por que isso é tão útil?

A vantagem monumental de converter a fórmula para usar a função max é a **independência das dimensões**.

Na fórmula original da soma, um x pequeno poderia ser compensado por um y grande, nos obrigando a olhar para o par sempre junto. Com a função max, podemos olhar para o eixo u isoladamente e para o eixo v isoladamente.

Voltando ao exemplo de encontrar a distância máxima entre N pontos: com o plano transformado, tudo o que precisamos fazer é encontrar a maior distância no eixo u (ou seja, o maior u menos o menor u) e a maior distância no eixo v (o maior v menos o menor v). O resultado final será simplesmente o maior desses dois valores.

Isso derruba a complexidade do problema de $O(N^2)$ para um rápido e eficiente $O(N)$.

mobius-inversion

Inversão de Möbius

A Inversão de Möbius é uma técnica poderosa na teoria dos números e combinatória, frequentemente utilizada para inverter somatórios sobre divisores ou para calcular a cardinalidade de conjuntos com propriedades de coprimalidade restritas.

Fórmula Geral (Forma de Divisores):

$$g(n) = \sum_{d|n} f(d) \iff f(n) = \sum_{d|n} \mu(d) g\left(\frac{n}{d}\right)$$

Onde $\mu(d)$ é a função de Möbius, definida como:

- $\mu(1) = 1$.
- $\mu(d) = 0$ se d possui um fator quadrado perfeito (i.e., $p^2 \mid d$).
- $\mu(d) = (-1)^k$ se d é o produto de k números primos distintos.

A propriedade fundamental que conecta essa função à contagem de MDC é dada por:

$$\sum_{d|\gcd(a,b)} \mu(d) = \begin{cases} 1, & \text{se } \gcd(a,b) = 1 \\ 0, & \text{se } \gcd(a,b) > 1 \end{cases}$$

Aplicação Clássica: Pares com MDC igual a x

Dado um vetor A de tamanho n com elementos positivos, o objetivo é contar quantos pares de índices (i, j) com $1 \le i, j \le n$ satisfazem $\gcd(A[i], A[j]) = x$.

Para resolver esse problema, definimos uma função auxiliar $C(v)$, que representa a quantidade de elementos no vetor A que são múltiplos de v.

Dedução da Fórmula: Queremos contar os pares (i, j) tais que $\gcd(A[i], A[j]) = x$. Isso equivale a exigir que $A[i]$ e $A[j]$ sejam múltiplos de x, e que ao dividi-los por x, eles sejam coprimos: $\gcd\left(\frac{A[i]}{x}, \frac{A[j]}{x}\right) = 1$.

Podemos escrever a quantidade total de pares desejados como:

$$\text{Pares} = \sum_{\substack{1 \le i \le n \\ x \mid A[i]}} \sum_{\substack{1 \le j \le n \\ x \mid A[j]}} \left[\gcd\left(\frac{A[i]}{x}, \frac{A[j]}{x}\right) = 1\right]$$

Substituindo a condição de coprimalidade pela propriedade fundamental de Möbius, temos:

$$\text{Pares} = \sum_{\substack{1 \le i \le n \\ x \mid A[i]}} \sum_{\substack{1 \le j \le n \\ x \mid A[j]}} \sum_{k \mid \gcd\left(\frac{A[i]}{x}, \frac{A[j]}{x}\right)} \mu(k)$$

A condição de que k divide o gcd de $\frac{A[i]}{x}$ e $\frac{A[j]}{x}$ significa que k divide $\frac{A[i]}{x}$ e k divide $\frac{A[j]}{x}$. Isso é matematicamente equivalente a dizer que $(k \cdot x)$ divide tanto $A[i]$ quanto $A[j]$.

Agora, invertemos a ordem dos somatórios. Em vez de iterar sobre os pares e depois sobre os divisores, iteramos sobre k (o multiplicador) e contamos quantos pares satisfazem a condição de serem múltiplos de $(k \cdot x)$.

Como o número de elementos em A que são múltiplos de $(k \cdot x)$ é $C(k \cdot x)$, o número de pares que podemos formar com esses elementos é simplesmente $C(k \cdot x)^2$. Agrupando tudo, chegamos à fórmula final.

Fórmula para Contagem de Pares:

$$\text{Pares} = \sum_{k=1}^{\lfloor M/x \rfloor} \mu(k) \cdot C(k \cdot x)^2 \pmod{10^9 + 7}$$

Onde M é o valor máximo presente no vetor A. A função $C(v)$ pode ser pré-computada para todos os valores em $O(M \log M)$ através de uma abordagem semelhante ao Crivo de Eratóstenes, permitindo responder à consulta para qualquer x em $O(M/x)$.

stars-and-bars

Stars and Bars

Problema base. Quantas soluções inteiras tem

$$x_1 + x_2 + \cdots + x_k = n, \qquad x_i \ge 0 \ ?$$

Ideia. Represente n como n estrelas ($\star$) e separe-as em k grupos usando $k-1$ barras ($|$). Cada arrumação de n estrelas e $k-1$ barras em fila corresponde a exatamente uma solução (o grupo i é a quantidade de estrelas entre a barra $i-1$ e a barra i). Ex.: $n = 5$, $k = 3$: $\star\star \mid \star \mid \star\star \leftrightarrow (2, 1, 2)$ e $\mid\mid \star\star\star\star\star \leftrightarrow (0, 0, 5)$.

Basta escolher as posições das barras entre $n + k - 1$ posições:

$$\boxed{\binom{n+k-1}{k-1}}$$

Variações

- **Variáveis positivas** ($x_i \ge 1$): coloque as barras nos $n-1$ espaços entre estrelas, no máximo uma por espaço:
$$\binom{n-1}{k-1}.$$
- **Limite inferior** ($x_i \ge l_i$): substitua $y_i = x_i - l_i \ge 0$. Recai no caso base com $n' = n - \sum l_i$ (se $n' < 0$, a resposta é 0):
$$\binom{n'+k-1}{k-1}.$$
- **Soma com desigualdade** ($x_1 + \cdots + x_k \le n$, $x_i \ge 0$): adicione uma variável de folga $x_{k+1} = n - \sum x_i \ge 0$. Agora são $k+1$ variáveis com soma exatamente n:
$$\binom{n+k}{k}.$$
- **Limite superior** ($0 \le x_i \le u_i$): inclusão-exclusão. Para um conjunto S de variáveis que *violam* o limite, force $x_i \ge u_i + 1$ para $i \in S$:
$$\sum_{S \subseteq \{1,\ldots,k\}} (-1)^{|S|} \binom{n - \sum_{i \in S}(u_i+1) + k - 1}{k-1},$$
considerando o termo 0 quando o topo fica negativo.
- **Mesmo limite superior** ($0 \le x_i \le c$ para todo i): os conjuntos S de mesmo tamanho contribuem igual, então
$$\sum_{j=0}^{k} (-1)^j \binom{k}{j} \binom{n - j(c+1) + k - 1}{k-1},$$
parando quando $n - j(c+1) < 0$.

Problemas equivalentes

A mesma fórmula $\binom{n+k-1}{k-1} = \binom{n+k-1}{n}$ conta:

- distribuir n objetos **idênticos** em k caixas **distintas**;
- **multiconjuntos** de tamanho n com elementos de k tipos (combinação com repetição);
- sequências **não-decrescentes** de tamanho n com valores em $[1, k]$ (x_i = quantas vezes o valor i aparece);
- monômios de grau n em k variáveis.

vandermonde-identity

Identidade de Vandermonde

A Identidade de Vandermonde é uma ferramenta essencial em combinatória para otimizar somatórios envolvendo multiplicações de coeficientes binomiais, reduzindo cálculos de complexidade $O(N)$ para $O(1)$ (com pré-computação de fatoriais).

Fórmula Clássica

Para quaisquer inteiros não negativos m, n e r:

$$\sum_{k=0}^{r} \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$$

Intuição Combinatória: Escolher um comitê de r pessoas de um grupo contendo m homens e n mulheres é equivalente a somar todos os cenários possíveis de escolha (onde escolhemos k homens e $r-k$ mulheres).

Uso em Programação Competitiva (O Truque da Simetria)

Em problemas de contagem, raramente a fórmula aparece pronta. O cenário mais comum é encontrar uma soma onde os índices inferiores **crescem simultaneamente**.

Para aplicar Vandermonde, a soma dos termos inferiores deve ser **constante**. Alcançamos isso aplicando a simetria do binômio:

$$\binom{n}{k} = \binom{n}{n-k}$$

Padrão 1: Índices Iguais

Se a sua modelagem resultou na soma:

$$\sum_{k=0}^{\min(x,y)} \binom{x}{k} \binom{y}{k}$$

Aplique a simetria no primeiro termo ($\binom{x}{k} \to \binom{x}{x-k}$):

$$\sum_{k} \binom{x}{x-k} \binom{y}{k} = \binom{x+y}{x}$$

Nota: A soma dos índices inferiores agora é $(x-k)+k = x$ (constante).

Padrão 2: Índices Deslocados (Exemplo Prático)

Se você precisar escolher $k-1$ itens de um grupo e k de outro:

$$\sum_{k \ge 1} \binom{x}{k-1} \binom{y}{k}$$

Transforme o primeiro termo usando $\binom{x}{k-1} = \binom{x}{x-(k-1)} = \binom{x}{x-k+1}$:

$$\sum_{k \ge 1} \binom{x}{x-k+1} \binom{y}{k} = \binom{x+y}{x+1}$$

Dica de Implementação: Sempre pré-compute os fatoriais e seus inversos modulares para responder a $\binom{n}{k} \pmod M$ em $O(1)$.